

HelmX-AI Powered Smart Helmet

Final Year Project – Final Report

Session 2025-2026

A Final Year Project submitted in partial fulfillment of
the requirements for the Degree
of
BS in Computer Science



Department of Computer Science

University of the Punjab, Lahore

Usman Azfar **BCSF22M512**

Hamza Ahmad **BCSF22M536**

Abdul Hanan **BCSF22M505**

Awais Imtiaz **BCSF22M541**

Under the Supervision of: **Dr. Tayyaba Tariq**

Project Details

Project ID (for office use)				
Type of project	☐ Traditional ☒ Industrial ☐ Continuing			
Nature of project	☐ Development ☒ Research & Development			
Sustainable Development Goals(SDGs)	☒ Good Health and Well-Being ☐ Quality Education ☒ Industry, Innovation, and Infrastructure ☐ Gender Equality ☒ Decent Work and Economic Growth ☐ Climate Action			
Area of specialization	☒ Artificial Intelligence (AI) ☐ Blockchain ☐ Cybersecurity ☒ Data Science and Analytics ☐ Game Development ☒ Internet of Things (IoT) ☒ Natural Language Processing (NLP) ☒ Mobile App Development ☒ Web Development			
Project Group Members				
Sr.#	Reg. #	Student Name	Email ID	Signature
(i)	Group Leader	Usman Azfar	bcsf22m512@pucit.edu.pk	
(ii)		Abdul Hannan	bcsf22m505@pucit.edu.pk	
(iii)		Hamza Ahmad	bcsf22m536@pucit.edu.pk	
(iv)		Awais Imtiaz	bcsf22m541@pucit.edu.pk	
Declaration: The candidates confirm that the work submitted is their own and appropriate credit has been given where reference has been made to the work of others.				

Plagiarism Free Certificate

This is to certify that, I am Usman Azfar S/D/o Azfar Shafiq , group leader of FYP under registration no BCSF22M512 at the Computer Science Department, Punjab University , Lahore . I declare that my FYP proposal is checked by my supervisor and the similarity index is _____% that is less than 20%, an acceptable limit by HEC. The report is attached herewith as Appendix A.

Date: _____ Name of Group Leader: _____ Signature: _____

Name of Supervisor: _____ Co-Supervisor (if any): _____

Designation: _____ Designation: _____

Signature: _____ Signature: _____

HoD: _____ Signature: _____

Table of Contents

Chapter 1. Introduction	9
1.1 Introduction	10
1.2 Problem Statement	11
1.3 Proposed Solution	11
1.4 Main Objectives	12
1.5 Assumptions & Constraints	12
1.5.1 Assumptions	12
1.5.2 Constraints	13
1.6 Project scope	14
1.6.1 Inclusion:	14
1.6.2. Optional:	14
1.6.3. Exclusion:	15
1.7 Software Development Lifecycle Model	15
1.7.1 Development Approach	15
1.7.2 Justification for Agile Methodology	15
1.7.3 Development Phases and Structure	16
1.7.4 Ensuring Systematic Progress	17
Chapter 2. Requirement Analysis	18
2.1 Overview	19
2.2 Crash Detection Systems	19
2.2.1 Threshold-Based Approaches	19
2.2.2 Machine Learning Approaches	20
2.3 Drowsiness Detection Systems	20
2.3.1 Vision-Based Detection	20
2.3.2 Physiological Sensor-Based Detection	21
2.4 Dual Camera and IMU Sensor Systems	21
2.4.1 IMU Sensor Implementation	21
2.4.2 Dual Camera Systems	22
2.5 GPS Navigation Systems	22
2.5.1 Wearable Navigation Approaches	23
2.5.2 Helmet-Specific Implementation	23
2.6 Voice Control and Entertainment Systems	23
2.6.1 Voice Control Systems	23
2.6.2 Entertainment Systems	24
2.6.3 AI Assistant Gap	24
2.7 Security and Theft Detection	24
2.7.1 Authentication Systems	25
2.7.2 Theft Detection and Tracking	25
2.8 Web and Mobile Application Interfaces	25
2.8.1 Web Dashboard Limitations	25

2.8.2 Mobile Application Architectures	26
2.9 Research Gaps and HelmX Contributions	26
2.10 Consolidated Comparison Table	28
2.11 Stakeholders List	29
2.11.1 Primary Stakeholders	29
2.11.2 Secondary Stakeholders	31
2.12 Requirements Elicitation	35
2.12.1 Functional Requirements	35
2.12.2 Non-Functional Requirements	41
2.12.3 Requirements Traceability Matrix	46
2.13 Use Case Description	49
2.13.1 Use Case 1: Crash Detection and Emergency Alert	49
2.13.2 Use Case 2: Drowsiness Detection and Alert	51
2.13.3 Use Case 3: Voice-Controlled Navigation	52
2.13.4 Use Case 4: Ride Analytics Review	54
Chapter 3. System Design	56
3.1 Use Case Design	57
3.2 System Architecture Diagram	58
Chapter 4. System Testing	59
4.1 Test Cases Design	60
4.1.1 Crash Detection Test Cases	60
4.1.2 Drowsiness Detection Test Cases	61
4.1.3 Smart Navigation Test Cases	62
4.1.4 Dual Camera System Test Cases	63
4.1.5 Voice Control and Entertainment Test Cases	64
4.1.6 Safety and Environment Monitoring Test Cases	65
4.1.7 Mobile and Web Application Test Cases	65
4.1.8 Security and Authentication Test Cases	66
4.2 Unit / Integration / Acceptance Testing	67
4.2.1 Unit Testing	67
4.2.2 Integration Testing	69
4.3 User Acceptance Testing	71
4.3.1 Test Participants	71
4.3.2 Testing Phases	71
4.4 Success Criteria (Summary)	72
4.5 Feedback Handling	73
4.6 Expected Outcomes	73
Chapter 5. Implementation	74
5.1 Work Breakdown Structure (WBS)	75
2.3 AI Companion System	75
4.2 Navigation System	76

4.3 AI Companion Integration	76
5.2 Dual Camera System	77
5.3 IMU & Sensor System	77
5.4 Theft Detection System	77
5.2 Team Roles and Responsibilities	78
5.3 Gantt Chart	79
5.4 Tools and Technologies	79
5.4.1 Machine Learning Development	79
5.4.2 Web Application Development	80
5.4.3 Mobile Application Development	80
5.4.4 Hardware Components (Procured)	80
5.4.5 Development and Collaboration Tools	81
5.5 Implementation Details	81
5.5.1 Crash Detection Model Implementation	81
5.5.2 Drowsiness Detection Model Implementation	82
5.5.3 Web Application Implementation	83
5.5.4 Mobile Application Implementation	84
5.5.5 Dual Camera System Implementation	85
5.5.6 IMU and Sensor System Implementation	86
5.5.7 AI Companion System Implementation	86
5.5.8 Theft Detection Implementation	87
5.5.9 Documentation Implementation	88
5.6 Screenshots of Prototype / System	89
5.6.1 Crash Detection Model	89
5.6.2 Drowsiness Detection Model	90
5.6.3 Web Application	91
5.6.4 Mobile Application	92
5.6.5 Hardware Components	94
5.7 Challenges During Implementation	95
5.7.1 Machine Learning Model Challenges	95
5.7.2 Web Application Challenges	96
5.7.3 Mobile Application Challenges	96
5.7.4 Hardware and Integration Challenges	97
5.7.5 Team Coordination Challenges	97
5.7.6 External Factors	98
Chapter 6. System Conversion	99
6.1 Introduction	100
6.2 Conversion Method	100
6.2.1 Justification:	100
6.2.2 Implementation Phases:	100
6.3 Deployment	100

6.3.1 Deployment Steps:	100
6.3.2 Data Conversion	101
6.3.3 Training	101
6.4 Post Deployment Testing	101
6.5 Challenges	102
6.6 Summary	102
Chapter 7. Cost Analysis	103
7.1 Introduction	104
7.2 Hardware Cost Breakdown	104
7.3 Development and Additional Costs	105
7.4 Total Cost Estimation	105
7.5 Cost Analysis and Feasibility	106
7.6 Summary	106
Chapter 8. Conclusion	107
8.1 Overview	108
8.2 Achievements	108
8.3 Limitations	108
8.4 Future Work	108
8.5 Final Remarks	109
Chapter 9. References	110

List of Tables

Table 1. Consolidated Comparison Table.....	28
Table 2. Stakeholder Tables	29-35
Table 3. Functional Requirement Tables.....	36-41
Table 4. Non Functional Requirement Tables.....	41-45
Table 5. Requirement Traceability Matrix.....	46-49
Table 6. Use Case Description Tables.....	49-54
Table 7. Team Roles and Responsibilities.....	78
Table 8. System Conversion Challenges.....	102
Table 9. Cost Analysis Tables.....	104-105

List of Figures

Figure 1: Use Case Design Diagram..... 57

Figure 2: System Architecture Diagram.....58

Figure 3: Gantt Chart.....79

Figure 4: Screenshots of Prototype/System89-94

Chapter 1. Introduction

1.1 Introduction

Motorcycle riders face disproportionately high risks on roadways, with the World Health Organization reporting that traffic accidents claim approximately 1.35 million lives globally each year. In Pakistan, more than 10,000 deaths occur annually from road accidents, with two-wheeled vehicles frequently involved. Motorcyclists are 28 times more likely to experience fatal accidents compared to automobile occupants due to limited physical protection, reduced visibility, and the cognitive demands of simultaneous vehicle operation and environmental awareness. When accidents occur, riders are often rendered immobilized or unconscious, unable to call for help during crucial minutes that determine survival outcomes. Additionally, rider fatigue and drowsiness contribute substantially to accident causation, yet motorcycle riders lack access to the drowsiness detection technology increasingly common in passenger vehicles.

The HelmX project addresses these critical safety challenges by developing an AI-powered smart helmet system that transforms passive protective equipment into an active safety ecosystem. The motivation stems from recognizing that existing solutions remain fragmented—offering either basic crash alerts, simple communication features, or isolated navigation assistance—but never integrating these capabilities into a comprehensive, affordable platform suitable for developing markets where motorcycle usage is most prevalent. This project holds significance beyond individual safety improvements, with implications for urban mobility in developing economies where motorcycles serve as primary transportation for millions. From an educational perspective, HelmX provides comprehensive learning opportunities across hardware integration, machine learning deployment, cloud infrastructure, and application development within a unified system addressing real-world problems with measurable social impact.

The primary objective is to create an intelligent helmet ecosystem that proactively prevents accidents through real-time monitoring while ensuring rapid automated response when incidents occur. This breaks down into five specific technical objectives: assembling a helmet prototype with integrated IMU sensors, cameras, GPS modules, and environmental sensors; developing and training machine learning models for crash and drowsiness detection achieving greater than 85% accuracy; implementing smart navigation and voice control systems that reduce cognitive load; building secure cloud infrastructure using AWS IoT Core and MongoDB for data management and emergency alerts and developing Android and web applications for ride analytics and system configuration.

The project scope includes seven core functionalities: crash detection with automatic emergency alerts, drowsiness detection with real-time warnings, smart navigation with traffic and weather considerations, dual camera systems for evidence collection, voice control and entertainment features, environmental monitoring with health recommendations, and comprehensive applications for analytics. Optional features such as gesture controls, posture monitoring, and augmented reality displays will be pursued only if primary objectives are completed successfully. The scope explicitly excludes large-scale manufacturing, medical-grade validation, and international safety certifications due to resource and timeline constraints.

HelmX employs a four-layer architecture integrating hardware sensors, edge AI processing using TensorFlow for real-time detection, cloud services for emergency

coordination and data storage, and user applications for monitoring and control. The development follows Agile methodology with two to four week sprints over eight to nine months, prioritizing core safety features before enhancement capabilities. Success will be measured through objective criteria including detection accuracy exceeding 85%, emergency alert transmission within ten seconds, GPS accuracy within ten meters, and System Usability Scale scores above 70 from pilot testing.

The main contributions include: an integrated safety ecosystem combining proactive prevention and reactive emergency response; edge-optimized AI deployment demonstrating real-time computer vision on resource-constrained hardware; context-aware emergency response eliminating human action requirements during critical post-accident seconds; an affordable solution template for developing markets using open-source frameworks; a multi-disciplinary integration methodology for complex systems development in academic settings; and an open research platform through comprehensive documentation. These contributions bridge academic research with practical implementation, demonstrating that sophisticated safety technology can be made accessible for vulnerable road users in resource-constrained environments.

1.2 Problem Statement

Motorcycle riders account for a disproportionately high number of global traffic fatalities, with Pakistan reporting over 10,000 annual deaths from road accidents involving two-wheeled vehicles. Motorcyclists are 28 times more likely to experience fatal accidents compared to car occupants due to limited visibility, rider fatigue, cognitive overload during navigation, and delayed emergency response when riders are immobilized. Existing solutions such as standard helmets and basic Bluetooth devices offer fragmented features but lack an integrated intelligent system capable of proactive accident prevention and automated emergency response. Current commercial systems either focus on isolated features with limited AI integration, are prohibitively expensive for developing markets, or fail to provide comprehensive data analytics. The absence of a unified, cost-effective solution that combines drowsiness detection, crash detection, smart navigation, and emergency alerts leaves a critical gap in motorcycle safety technology for low-resource communities in developing nations.

1.3 Proposed Solution

The HelmX system addresses the identified safety challenges through an integrated AI-powered smart helmet ecosystem that combines real-time monitoring, intelligent feedback, and automated emergency response. The solution employs a multi-layered architecture comprising a hardware layer with embedded sensors (accelerometers, gyroscopes, dual cameras, GSM module), an edge AI layer for on-device processing using TensorFlow models, a cloud layer built on AWS IoT Core with MongoDB for data management, and an application layer featuring React.js web dashboard and Kotlin-based Android app. The system implements seven core functionalities: crash detection utilizing IMU sensors with machine learning classifiers achieving >85% accuracy to trigger automatic GPS-based emergency alerts; drowsiness detection through computer vision algorithms using OpenCV and CNNs to analyze eye closure for real-time audio-visual warnings; smart navigation leveraging Google Maps API with AI-based route optimization considering

traffic and weather conditions; dual camera system for continuous evidence recording with voice-commanded capture; voice control and entertainment system using NLP for hands-free operation adaptable to ambient noise; safety and environment monitoring measuring air quality, temperature, and allergens to generate ride recommendations; and comprehensive web and Android applications providing ride analytics, historical data visualization, and system configuration. Unlike existing fragmented solutions, HelmX differentiates itself through seamless integration of safety features into a single cost-effective platform utilizing open modular design principles, making it accessible for developing markets. The edge computing approach minimizes latency for critical safety functions while cloud integration enables sophisticated analytics and emergency response coordination. This unified ecosystem reduces cognitive load on riders through intelligent assistance while maintaining situational awareness, ultimately aiming to reduce accident risks significantly through proactive prevention and rapid post-incident response. The expected outcomes include a functional prototype with >85% detection accuracy in user testing, positive System Usability Scale scores (>70) from pilot users, and a scalable architecture that can be adapted for broader deployment in urban mobility systems across developing nations.

1.4 Main Objectives

The main objectives of this project are:

- To design and assemble a helmet prototype with integrated sensors (IMU, cameras, microphone).
- Integrate dual cameras for evidence capture and voice-commanded media.
- To develop and train AI models for drowsiness and crash detection.
- To implement a real-time navigation system, voice control system and safety system.
- To build a secure cloud backend for data processing and alerts.
- To integrate a personalized AI peer for communication and entertainment during long tours
- Establish environment monitoring for air quality, temperature, weather which then provides related suggestions.
- To develop an Android app and a responsive web dashboard.

1.5 Assumptions & Constraints

1.5.1 Assumptions

The HelmX project operates under several fundamental assumptions that define the operational context and expected conditions for system functionality:

- **Hardware Availability:** All required hardware components including cameras, IMU sensors, GPS modules, microphones, and edge computing devices are commercially available and can be procured within the project timeline and budget.
- **Network Connectivity:** Riders will have access to mobile network connectivity (3G/4G/5G) for cloud communication, emergency alerts, and real-time navigation services in most operational areas.

- **User Cooperation:** Test riders and end-users will provide honest feedback, wear the helmet correctly, and follow safety protocols during testing and deployment phases.
- **Environmental Conditions:** The system will operate under normal riding conditions with standard weather variations, excluding extreme environmental scenarios such as heavy storms or extreme temperatures beyond -10°C to 50°C.
- **Data Availability:** Sufficient training data for drowsiness detection and crash detection models is available through public datasets or can be collected during the development phase.
- **Power Supply:** The integrated battery system can provide adequate power for continuous operation of all sensors and processing units for a minimum of 2-3 hours of active riding.
- **GPS Accuracy:** GPS services will provide location accuracy within 5-10 meters for emergency alert purposes and navigation guidance in most geographic locations.
- **Cloud Service Reliability:** AWS cloud services will maintain 99.9% uptime and provide consistent performance for data storage, processing, and alert transmission.

1.5.2 Constraints

The project development and deployment are subject to the following technical, operational, and resource-based constraints:

- **Budget Limitations:** The project must be completed within the estimated project budget, restricting the use of expensive commercial-grade sensors and limiting the number of physical prototypes to one or two units.
- **Time Constraints:** Development must be completed within the academic session timeline of 8-9 months (September 2025 - May 2026), requiring prioritization of core features over optional functionalities.
- **Processing Power:** Edge computing capabilities are limited by the size, weight, and power consumption requirements of helmet-mounted devices, restricting the complexity of on-device AI models and requiring optimization through TensorFlow Lite.
- **Storage Capacity:** On-device storage for video recording and data logging is limited to available memory on edge computing units, necessitating efficient data compression and cloud offloading strategies.
- **Battery Life:** The system must balance power consumption across multiple sensors and processing units while maintaining reasonable battery life, constraining the frequency of sensor readings and video recording quality.
- **Network Latency:** Real-time emergency alerts and cloud communication are subject to network latency and bandwidth limitations, particularly in rural or poorly connected areas.
- **Scope Limitations:** The project excludes large-scale hardware manufacturing, medical-grade validation of health metrics, advanced hardware features like dynamic visor tinting, and international standard certifications due to resource and expertise constraints.
- **Technical Expertise:** Development is constrained by the team's current skill level in embedded systems and hardware integration, requiring significant learning and adaptation during the project lifecycle.

- **Data Privacy and Security:** The system must comply with basic data protection practices while operating within the constraints of available security frameworks and without formal legal certification for data handling.
- **Camera Resolution and Quality:** Video recording quality is constrained by the balance between resolution, storage requirements, and processing power, limiting continuous recording to standard definition or compressed high-definition formats.

1.6 Project scope

The project's scope includes the end-to-end development of the HelmX system. Deliverables include the physical prototype, all source code for the firmware, AI models, mobile app, and web dashboard, along with full documentation.

1.6.1 Inclusion:

In-scope features are the seven core functionalities:

- Smart crash detection system to indicate sudden movements or abnormal movements indicative of crash and automatically triggers alerts to predefined contacts with location data upon crash detection.
- Drowsiness Detection system using camera that detects drowsiness and issues real time alerts.
- Integrates GPS and smart navigation for real-time route guidance and suggestions.
- Features dual cameras and IMUs for comprehensive environmental awareness. Supports real-time video recording, live streaming, or image capture for safety documentation and situational analysis.
- Enables hands-free operation through voice recognition for controlling navigation, communication, or media playback. Includes an integrated entertainment system for audio streaming (e.g., music, podcasts) via Bluetooth- connected devices. It may also include a real time AI peer as a ride companion.
- For safety and security, it will include proper authentication and authorization and will generate real time alerts if theft is detected.
- Complete web application and android app application for real time analytics and control

1.6.2. Optional:

Several "Optional" features can also be included if time permits. These include:

- Gesture Control
- Posture Detection and suggestion
- Smart Sound Controlling according to traffic
- Distance Monitoring and Fuel Estimation system
- Smart Rider Feedback system
- Smart Public Awareness System of dangerous areas on road
- Solar Assisted Charging system
- Heads up Display with Augmented Reality
- Smart Speed Optimization System for constant speed with less brakes

- AI bike scanner and Tutorials provider to fix issues
- Smart Personalized Riding Coach

1.6.3. Exclusion:

Out-of-scope items include:

- Hardware manufacturing at scale
- Advanced hardware like dynamic tinting
- Medical-grade validation of health metrics
- Smart mesh network system with nearby riders
- Smart gamified safe ride score
- Smart battery optimization
- Research on effect of electromagnetic field from sensors and microcontrollers on human health.

1.7 Software Development Lifecycle Model

1.7.1 Development Approach

The HelmX project adopts the **Agile Scrum methodology** as its primary software development lifecycle model, implemented through iterative sprints of 2-4 weeks duration over the 8-9 month development period. This approach is particularly well-suited for the multi-disciplinary nature of the project, which integrates hardware prototyping, AI model development, cloud infrastructure, and application development across multiple platforms.

1.7.2 Justification for Agile Methodology

The selection of Agile methodology is justified by several critical factors:

Iterative Development Requirements: The project involves experimental AI models for drowsiness and crash detection requiring continuous refinement based on testing results. Agile's iterative nature allows for rapid prototyping and adjustment of machine learning algorithms to achieve the target >85% accuracy.

Hardware-Software Integration Complexity: The integration of IoT sensors, embedded systems, cloud services, and mobile applications necessitates frequent integration testing. Agile sprints enable early identification and resolution of integration issues.

Evolving Requirements: As an R&D project exploring emerging technologies, requirements may evolve based on hardware capabilities and user feedback. Agile accommodates requirement changes between sprints without disrupting the development timeline.

Multi-Component Parallel Development: With four team members working on different components, Agile facilitates parallel development while ensuring regular integration through sprint reviews.

Risk Mitigation: The 8-9 month timeline with multiple interdependent components carries inherent risks. Agile's incremental delivery ensures core functionalities are developed and tested first, allowing prioritization of critical features if time constraints emerge.

1.7.3 Development Phases and Structure

The Agile implementation for HelmX is structured across the following phases, executed iteratively within 2-4 week sprints:

1.7.3.1 Sprint 0: Project Initialization (Weeks 1-2)

- Conduct literature review and define functional and non-functional requirements
- Design four-layer system architecture and define component interfaces
- Procure hardware components and set up development environments
- Create product backlog with prioritized user stories

1.7.3.2 Sprints 1-3: Core Safety Features (Weeks 3-10)

- Assemble helmet prototype with IMU sensors, cameras, GPS module, and battery system
- Develop and train crash detection ML classifier with TensorFlow
- Implement drowsiness detection CNN model using OpenCV for real-time video processing
- Set up AWS IoT Core, MongoDB database, and mechanism for emergency alerts
- Validate detection accuracy and emergency alert functionality

1.7.3.3 Sprints 4-6: Navigation and Monitoring (Weeks 11-18)

- Integrate Google Maps API/OpenMapStreetMap
- Configure dual cameras for continuous recording and voice-command capture
- Develop NLP-based voice recognition system.
- Integrate environmental sensors for air quality and temperature monitoring.
- Test navigation accuracy, voice recognition, and sensor reliability

1.7.3.4 Sprints 7-9: Application Development (Weeks 19-26)

- Develop Kotlin-based Android app with real-time data display and ride analytics
- Build responsive React.js web dashboard
- Develop RESTful APIs using Node.js for helmet-cloud-application communication
- Conduct end-to-end system integration testing
- Validate real-time data flow and emergency alert workflow

1.7.3.5 Sprints 10-12: Testing and Deployment (Weeks 27-36)

- Optimize ML model inference time, reduce latency, and improve battery efficiency
- Conduct user acceptance testing with test rider
- Validate detection accuracy (>85%) and false positive rates (<10%)
- Address bugs and refine features based on real-world testing
- Complete technical documentation, user manual, and research paper
- Deploy final prototype, web dashboard, and Android application

1.7.3.6 Continuous Activities

- Daily 15-minute stand-up meetings for team synchronization
- Version control with Git, code reviews, and branch management
- Unit and integration testing with continuous regression testing
- Weekly progress updates to supervisor with bi-weekly demonstrations

1.7.4 Ensuring Systematic Progress

The Agile approach ensures systematic progress through several integrated mechanisms:

Sprint Planning and Backlog Management: Each sprint begins with planning sessions where user stories are selected, decomposed into tasks, and assigned based on team expertise.

Definition of Done: Clear acceptance criteria including functional requirements (>85% accuracy), performance benchmarks (<2 second latency), and quality standards (code review, test coverage).

Velocity Tracking and Burndown Charts: Team velocity is measured to enable accurate estimation of remaining work, with visual tracking helping identify delays early.

Continuous Integration: Automated build processes ensure code from different team members can be merged and tested frequently, reducing integration risks.

Sprint Retrospectives: End-of-sprint meetings identify improvements in processes, communication, and technical approaches for subsequent sprints.

Incremental Deliverables: Each sprint produces working system increments that can be demonstrated and tested, ensuring tangible progress.

Adaptability: The framework allows reprioritization of optional features if core functionalities require more time, ensuring successful delivery of the minimum viable product within the academic timeline.

This structured yet flexible approach ensures HelmX maintains systematic progress while accommodating uncertainties in hardware-software integration, AI model development, and user experience refinement required for a comprehensive motorcycle safety system.

.

Chapter 2. Requirement Analysis

Literature Review

2.1 Overview

Smart helmet technology represents a convergence of artificial intelligence, Internet of Things, computer vision, and wearable computing aimed at enhancing motorcycle rider safety. This literature review examines existing research and commercial implementations across seven critical domains: crash detection systems, drowsiness detection, dual camera and IMU sensor integration, GPS navigation, voice control and entertainment systems, security and theft detection, and mobile/web application interfaces. The review identifies gaps in current approaches and establishes the foundation for HelmX's innovative contributions.

2.2 Crash Detection Systems

Motorcycle crashes represent one of the most critical safety challenges globally, with motorcyclists being approximately 28 times more likely to die in traffic accidents compared to passenger car occupants [1]. Rapid emergency response following crashes significantly improves survival rates, with studies indicating that reducing emergency medical services notification time by even minutes can decrease fatality rates by up to 30% [2].

2.2.1 Threshold-Based Approaches

Traditional crash detection systems have relied primarily on threshold-based approaches utilizing acceleration and velocity measurements. A comprehensive study on crash detection device development established a methodology utilizing two key parameters during a 100-millisecond time window: measured acceleration and velocity difference [3]. The system determined crash occurrence when both acceleration exceeded 1g and velocity difference corresponded to an average acceleration of 25% of the acceleration threshold. While demonstrating zero false positives under normal driving conditions, harsh stops triggered false alarms, revealing limitations in distinguishing severe braking from actual collisions.

Research on low-cost GPS/IMU integrated accident detection systems employed threshold-based filtering with deceleration measurements from Inertial Measurement Units [4]. The methodology implemented a two-stage verification process: initial detection triggered when deceleration exceeded 5g, followed by velocity confirmation requiring vehicle speed below 5 km/h to confirm crash occurrence. Testing demonstrated successful differentiation between false accidents and true accidents, validating the dual-threshold approach for reducing false alarms.

Lee et al. [5] compared threshold-based, supervised, and unsupervised approaches for crash detection in e-scooters. The threshold-based model employed three sequential criteria: Acceleration Sum Vector Magnitude (ASVM) threshold, Angular Velocity Sum Vector Magnitude (GSVM) threshold, and roll angle threshold satisfaction within 0.2 seconds. Optimal thresholds were determined through grid search: ASVM (5g to 20g),

GSVM (50°/s to 300°/s), and roll angle (20° to 50°). The model achieved lead times exceeding 250 milliseconds, sufficient for airbag deployment systems.

These threshold-based approaches exhibited critical limitations: fixed thresholds failed to adapt to varying riding conditions, high susceptibility to false positives during harsh braking, inability to distinguish between crash types, and no learning capability to improve detection accuracy over time.

2.2.2 Machine Learning Approaches

The e-scooter fall detection study implemented an unsupervised autoencoder model with knowledge distillation to address data imbalance challenges inherent in crash detection [5]. The architecture trained exclusively on normal driving data using a sliding-window technique (1-second windows, 75% overlap, 6×60 dimension input from 6-axis IMU at 60 Hz). The autoencoder learned normal riding patterns and identified crashes as anomalies when reconstruction error exceeded the maximum error observed during training. Knowledge distillation transferred learning from a larger teacher model to a smaller student model, achieving 98.86% accuracy with 96% memory reduction (46 KB vs. 1437 KB).

Research on automatic All-Terrain Vehicle crash detection developed a rollover detection system utilizing sensor fusion techniques [6]. The methodology employed a 3-axis IMU for attitude estimation fused with low-cost GPS for position tracking. A Madgwick filter combined accelerometer, gyroscope, and magnetometer data to reduce attitude estimation errors, while an Unscented Kalman Filter fused GPS and IMU data for improved position accuracy. The system achieved >99% detection accuracy with average EMS notification time of 41 seconds.

2.3 Drowsiness Detection Systems

Driver drowsiness contributes to approximately 20% of all traffic accidents globally and causes over 100,000 crashes annually in the United States alone [7]. For motorcycle riders, the consequences of drowsiness-related incidents are particularly severe due to reduced vehicle stability and minimal protective barriers.

2.3.1 Vision-Based Detection

Ahmed et al. [8] developed a deep learning approach to driver drowsiness detection using facial features. The system utilized Convolutional Neural Networks and VGG16 models to detect and classify facial expressions into four categories: open eyes, closed eyes, yawning, and no yawning. Testing on a dataset of 2,900 images achieved approximately 94% accuracy in controlled conditions.

Maier et al. [9] proposed a real-time classification system for autonomous drowsiness detection using the Eye Aspect Ratio (EAR) metric. The methodology operated in three main components: eye detection using computer vision, EAR calculation with blink classification, and real-time drowsiness assessment. The system detected long blinks and

eye closures as fatigue indicators, achieving approximately 93% accuracy on the DROZY database.

Magán et al. [10] developed an Advanced Driving Assistance System focused on drowsiness detection using deep learning techniques on 60-second video sequences. The research implemented two alternative solutions: a recurrent and convolutional neural network approach, and a hybrid system combining deep learning feature extraction with fuzzy logic. The system achieved 90-95% accuracy while emphasizing minimization of false positives for practical deployment.

These vision-based systems demonstrated effectiveness in controlled environments but exhibited several limitations: substantial computational requirements, processing latency challenges for real-time edge deployment, limited performance in varying lighting conditions, and high computational intensity for video sequence processing.

2.3.2 Physiological Sensor-Based Detection

Dhole et al. [11] proposed a novel helmet design incorporating EEG-based drowsiness detection specifically for industrial workers. Using Random Forest classifier algorithms on time and frequency domain features extracted from brainwave data, the system achieved over 98.6% accuracy in controlled laboratory settings. The helmet also integrated an IMU for fall detection, providing comprehensive safety monitoring.

Rosenberg et al. [12] developed and validated a comprehensive smart helmet system integrating multiple physiological sensors including EEG, ECG, and respiration monitoring capabilities. Electrodes were embedded at strategic facial locations within a standard helmet shell. The research validated signal quality during motion and active conditions, demonstrating the feasibility of continuous vital sign monitoring even in high-mobility environments.

While physiological sensor-based systems achieved high accuracy rates, they faced significant practical limitations: EEG sensors required direct skin contact causing discomfort, complex calibration requirements, high implementation costs, intrusive electrode placement affecting comfort, and specialized expertise requirements for deployment and maintenance.

2.4 Dual Camera and IMU Sensor Systems

The integration of dual camera systems and IMU sensors in smart helmets represents a critical advancement in motorcycle safety technology, serving dual purposes: providing comprehensive environmental awareness through visual documentation and enabling accurate crash detection through motion sensing.

2.4.1 IMU Sensor Implementation

The MPU6050 has emerged as the predominant IMU sensor in smart helmet applications due to its integration of both three-axis gyroscope and three-axis accelerometer in a single package. Cismas et al. [13] demonstrated that the MPU6050 IMU sensor can effectively

measure Euler angles (roll, pitch, yaw) and G-forces on different axes for crash detection in motorcycle applications. The sensor's built-in Digital Motion Processor minimizes gyroscopic drift and accelerometer errors, enhancing motion and data accuracy.

Wong et al. [14] conducted a comprehensive study on head motion recognition using a smart helmet integrated with IMU sensors, achieving classification accuracy ranging from 95.9% to 99.1% for different head motions. Their research revealed that gyroscope sensor data is more useful than accelerometer data alone for head motion recognition, highlighting the importance of sensor fusion approaches.

Despite widespread adoption, several limitations persist: limited processing capabilities with basic microcontrollers, absence of edge computing introducing latency issues, single-purpose design focusing primarily on crash detection, calibration challenges on resource-constrained devices, and environmental sensitivity to vibrations and magnetic disturbances.

2.4.2 Dual Camera Systems

The JARVISH X-AR represents one of the most advanced commercial smart helmet implementations, featuring 2K front cameras and 1080p 360-degree front and rear cameras [15]. The helmet utilizes ultra-wide-angle cameras capable of recording approximately a 360-degree panorama, with Alexa voice control for hands-free operation.

The Global Eagle FD-2 helmet camera features a lightweight front and rear cam design with 4K front and 1080p HD rear resolution, along with image stabilization [16]. However, this system operates as an external attachment rather than an integrated solution, compromising aerodynamics and requiring separate power management.

Academic research on helmet cameras has primarily focused on their utility as evidence collection tools rather than integrated safety systems. Helmet camera footage provides several benefits including encouraging driver honesty, adequately showing incident severity, and potentially lowering insurance rates [17].

Existing camera systems face limitations including: high power consumption (2-6 hours continuous recording), lack of intelligent storage management, limited research on optimal placement, absence of edge AI processing, camera quality concerns, weight and bulk issues, high cost (\$1,500+ for advanced systems), poor integration with other sensors, and manual activation requirements.

2.5 GPS Navigation Systems

Navigation assistance represents a critical safety feature for motorcycle riders, as traditional handlebar-mounted displays require riders to divert visual attention from the road. Studies indicate that visual distraction accounts for approximately 30% of motorcycle accidents, with navigation-related distraction being a significant contributing factor [18].

2.5.1 Wearable Navigation Approaches

Kan et al. [19] developed the eLabador system, an autonomous navigation solution combining GPS with RGB-D SLAM technology for visually impaired users. The system integrated live GNSS positioning with computer vision-based environment mapping to provide reliable navigation guidance. While demonstrating qualitative success, it exhibited significant power consumption and faced limitations in environments with poor visual features.

Xu et al. [20] proposed a phone-based navigation system utilizing remote agent technology with cloud connectivity. The architecture leveraged smartphone GPS capabilities combined with cloud-based route optimization and real-time traffic data integration. However, the system's dependence on continuous internet connectivity limited functionality in areas with poor cellular coverage.

2.5.2 Helmet-Specific Implementation

Elabd et al. [21] developed Helmify, a smart helmet navigation system utilizing ESP32 microcontroller with BLE communication and Google Maps API integration. The system received navigation instructions from a paired smartphone and translated them into visual indicators on the helmet's heads-up display. However, the study did not provide quantitative accuracy metrics or latency measurements.

Research on BLE communication latency in wearable devices [22] reported typical BLE latency ranging from 100-200 milliseconds under optimal conditions, with performance degradation in electromagnetically noisy environments. These findings established baseline expectations for wireless communication performance in helmet-based systems.

Current navigation systems exhibit limitations including: manual configuration requirements, connectivity dependence, GPS accuracy issues (± 5 -10 meters), signal loss in urban canyons, communication latency (200+ milliseconds), high power consumption, over-reliance on visual displays, and poor anticipatory guidance.

2.6 Voice Control and Entertainment Systems

The integration of voice recognition, entertainment systems, and artificial intelligence assistants in smart helmets represents a convergence of safety technology, human-computer interaction, and rider convenience.

2.6.1 Voice Control Systems

The Sena Stryker helmet incorporates voice command functionality integrated with Bluetooth 5.0 technology, allowing riders to control basic functions through predetermined voice commands [23]. However, voice recognition accuracy degrades significantly at speeds exceeding 30 mph due to wind noise interference.

Cardo's PACKTALK Edge series feature natural voice operation with the wake word "Hey Cardo," achieving voice command recognition rates of approximately 85% in controlled

environments but suffering performance degradation in real-world riding conditions with ambient noise levels exceeding 85 dB [24].

Lee et al. [25] conducted a comprehensive scoping review of smart helmets with multimodal sensing, revealing that current voice control systems primarily function as basic command interfaces rather than intelligent assistants. Key limitations include: acoustic challenges (wind noise generating 85-105 dB at highway speeds), limited natural language processing with keyword spotting only, lack of context awareness, latency issues (1.5-3 seconds), limited multilingual capabilities, single-task orientation, and no adaptive learning.

2.6.2 Entertainment Systems

The Sena 10R features 40mm HD speakers with advanced noise control and audio multitasking capabilities [26]. The system achieves sound pressure levels exceeding 115 dB, ensuring audibility even at highway speeds with ear protection.

Cardo's collaboration with JBL produced 40mm and 45mm speaker systems specifically engineered for helmet environments [27]. The PACKTALK Pro incorporates these speakers with Dynamic Mesh Communication technology, supporting up to 15 simultaneous rider connections.

Islam et al. [28] implemented an IoT-based smart helmet for riding security with basic Bluetooth audio capabilities. Their study revealed that riders frequently adjusted audio settings manually during rides, indicating the need for automated, context-aware audio management systems.

Limitations include: static audio management regardless of conditions, no real-time adaptation to rider profiles, basic entertainment features without intelligent content recommendation, and limited sophistication in managing audio priority between multiple sources.

2.6.3 AI Assistant Gap

Despite advances in AI technology, current smart helmet implementations remain limited to basic voice command interfaces connected to smartphone-based assistants. By 2024, over 8.4 billion digital voice assistants were in use globally [29], yet integration into motorcycle helmets remains superficial. Academic literature reveals a significant gap in research on conversational AI integration in motorcycle helmets, with existing studies focusing on accident detection and basic voice commands rather than true conversational AI capabilities.

2.7 Security and Theft Detection

Motorcycle theft represents a significant economic and security concern globally, with over 188,500 motorcycles stolen annually in the United States alone [30]. Smart helmets present unique opportunities for implementing theft detection and security features.

2.7.1 Authentication Systems

Preetham et al. [31] developed a safety helmet with alcohol detection and theft control using face recognition for rider authentication. The system employed the Viola-Jones face detection algorithm coupled with Support Vector Machines for classification, achieving approximately 85% accuracy in controlled lighting conditions. However, recognition accuracy degraded significantly in low-light conditions (below 60%), and processing latency of 3-5 seconds made real-time authentication impractical.

Sudarsan and Diderot [32] proposed a head protector with wireless bike authentication using RFID technology. The system verified the helmet's proximity to the motorcycle before allowing engine ignition. Kalkotwar et al. [33] extended this concept in their IoT-based smart helmet, implementing Bluetooth-based authentication between helmet and vehicle.

Research by Jain et al. [34] demonstrated that multimodal biometric systems combining face and fingerprint recognition achieve superior performance (>95% accuracy) compared to single-modality systems. However, no peer-reviewed studies have implemented comprehensive multimodal authentication specifically in smart helmet environments.

2.7.2 Theft Detection and Tracking

Multiple research efforts have integrated GPS tracking into smart helmets. Chandran et al. [35] developed "Konnnect," an IoT-based smart helmet providing accident detection and GPS location sharing. Priya et al. [36] implemented a smart bike helmet with vehicle tracking using Arduino, incorporating GPS modules for continuous location monitoring.

Commercial motorcycle GPS tracking solutions like Monimoto 9, BikeTrax, and AlterLock provide comprehensive anti-theft features including real-time tracking, geofencing, movement alerts, and theft recovery modes [37]. These systems achieve positioning accuracy within 5-10 meters and offer 365-day location history storage.

Current systems exhibit limitations including: single-factor authentication vulnerability, no continuous verification, limited scalability for multiple users, weak security protocols, no tamper detection, insufficient GPS accuracy in urban canyons, high battery consumption, high false alarm rates (25-35%), and lack of contextual awareness.

2.8 Web and Mobile Application Interfaces

The integration of mobile applications and web interfaces with smart helmet systems has evolved significantly, though the application components remain critical for user interaction, data visualization, and system control.

2.8.1 Web Dashboard Limitations

Current smart helmet web dashboards predominantly utilize traditional 2D layouts for data visualization. Sharma et al. [38] implemented a smart dashboard with graphical touch screen interface for helmet-based authentication systems, providing basic features such as

maintaining service records and configuring helmet settings. However, the interface lacked advanced visualization capabilities and offered limited user customization options.

Rathod [39] incorporated cloud dashboard services using Thinger.io platform, featuring time-series graphs for temperature values and GPS location visualization. While functional, the dashboard utilized static 2D representations that did not provide users with an intuitive understanding of the helmet's physical configuration.

Kim et al. [40] identified several critical limitations in existing web interfaces: non-interactive design serving as passive information displays, static 2D visualization failing to leverage modern 3D technologies, limited customization options with predefined packages, absence of real-time 3D preview, poor mobile responsiveness, and absence of dynamic cost calculation.

2.8.2 Mobile Application Architectures

Alcantara et al. [41] developed an IoT-based smart helmet with a mobile application built using Bubble.io and Firebase, achieving 90% accuracy in crash detection. However, their application interface was limited to basic notification systems and lacked real-time analytics visualization capabilities.

Kumar et al. [42] utilized Android smartphones with Firebase real-time database integration. While successfully demonstrating accident detection, the application interface was rudimentary, offering minimal user customization and lacking sophisticated data visualization features.

Commercial implementations show persistent limitations. The iC-R smart helmet features a companion mobile app for both Android and iOS platforms [43], but user reviews reveal static 2D layouts lacking immersive visualization. Sena Technologies' Motorcycles App provides remote control for various features [44], but user feedback from app stores reveals persistent connectivity issues and non-intuitive interfaces with difficulties in Bluetooth pairing and firmware updates [45][46].

Research indicates that native Kotlin development offers superior performance for real-time data processing requirements typical in smart helmet applications [47][48]. However, most commercial applications are developed using Java or hybrid frameworks like React Native and Flutter, resulting in suboptimal performance. Additional limitations include poor integration of modern UI/UX trends, limited real-time analytics dashboards, and inefficient handling of high-frequency sensor data streams.

2.9 Research Gaps and HelmX Contributions

Based on comprehensive literature analysis, the following critical gaps exist across all reviewed domains:

Integrated Systems Gap: No current system effectively combines high-resolution dual cameras, advanced IMU sensors, edge AI processing, GPS navigation, voice control, and

security features in a single, integrated smart helmet platform accessible to developing markets.

Edge Computing Deficit: Most implementations rely heavily on cloud-based processing, introducing latency issues and requiring constant internet connectivity. The lack of edge computing capabilities prevents real-time decision-making at the device level for safety-critical functions.

Affordability Barrier: Advanced smart helmets like JARVISH X-AR cost \$1,500+, excluding the majority of global motorcycle riders, particularly in developing nations where the need is greatest.

Application Interface Limitations: Existing mobile and web interfaces lack modern 3D visualization, component-level customization, real-time analytics, and intelligent user experiences.

AI Integration Gap: Despite advances in conversational AI, no smart helmet integrates true multimodal AI assistants capable of natural dialogue, contextual understanding, and proactive assistance during riding.

Security Vulnerabilities: Current authentication systems rely on single-factor methods vulnerable to bypassing, with no continuous verification or comprehensive anti-theft features.

HelmX addresses these gaps through innovative integration of: unsupervised learning-based crash detection with knowledge distillation for edge deployment achieving >85% accuracy, MediaPipe-based drowsiness detection with multi-feature fusion, Raspberry Pi 4 edge computing platform for on-device processing, automatic GPS navigation with <150ms latency, voice recognition, interactive 3D web configurator with component-level customization, high-performance Kotlin Android application with comprehensive real-time analytics, and cost-optimized modular design with pay-as-you-grow pricing (\$150-400 vs. \$1,500+ commercial alternatives).

2.10 Consolidated Comparison Table

Feature/System	Crash Detection	Drowsiness Detection	Camera/IMU	Navigation	Voice/Entertainment	Security	Mobile/Web App
Threshold Systems [3][4]	85-92% accuracy, high false positives	Not implemented	Basic IMU only	Not applicable	Not applicable	Not applicable	Not applicable
E-Scooter Autoencoder [5]	98.86% accuracy, 46KB model	Not implemented	6-axis IMU	Not applicable	Not applicable	Not applicable	Not applicable
Ahmed et al. [8]	Not implemented	94% accuracy, CNN+VG G16	Camera only	Not applicable	Not applicable	Not applicable	Not applicable
Dhole et al. [11]	Not implemented	98.6% EEG-based	IMU for falls	Not applicable	Not applicable	Not applicable	Not applicable
JARVISH X-AR [15]	Basic accelerometer	Not implemented	2K front, 1080p rear	Not specified	Alexa integration	None	Basic 2D app
Sena Systems [23][26]	Not implemented	Not implemented	Not implemented	Not implemented	Bluetooth, 85% voice accuracy	None	Limited app
Elabd Helmify [21]	Not implemented	Not implemented	Not specified	BLE + Google Maps	Not implemented	Not implemented	Basic interface
Preetham et al. [31]	Not implemented	Not implemented	Face recognition camera	Not implemented	Not implemented	85% face auth	Not specified
Commercial GPS [37]	Not applicable	Not applicable	Not applicable	GPS tracking	Not applicable	Basic geofencing	Mobile app
HelmX (Proposed)	>95% unsupervised ML, <50KB	94-96% MediaPipe + MobileNetV2	4K front, 2K rear + MPU6050	<150ms BLE, automatic	Gemini AI integration	Multimodal biometric > 98%	3D web + Kotlin app

Table 1: Consolidated Comparison Table

2.11 Stakeholders List

This section identifies all individuals, groups, and organizations who have a direct or indirect interest in the HelmX project. Each stakeholder plays a specific role and has varying levels of influence and involvement in the project's development, deployment, and outcomes.

2.11.1 Primary Stakeholders

2.11.1.1 Development Team

Attribute	Details
Members	Usman Azfar (Group Leader), Hamza Ahmad, Abdul Hannan, Awais Imtiaz
Role	Core development team responsible for system design, implementation, testing, and deployment of HelmX
Responsibilities	Requirements analysis, system architecture design, coding, integration of AI/IoT modules, testing, debugging, and final deployment
Involvement Level	High – actively involved throughout all phases of the project lifecycle
Key Interests	Successful project completion, application of AI, IoT, cloud and full-stack skills, academic achievement
Influence Level	High – full control over technical decisions, feature prioritization, and timelines
Impact on Project	Directly determines project success, quality, and innovation level

2.11.1.2 Project Supervisor

Attribute	Details
Name	Dr. Tayyaba Tariq
Role	Academic supervisor providing mentorship, guidance, and evaluation
Responsibilities	Reviewing progress, approving milestones, ensuring research quality, providing technical feedback
Involvement Level	High – regular meetings, sprint reviews, and evaluations
Key Interests	Academic rigor, quality deliverables, student learning outcomes, research contribution
Influence Level	High – authority over approvals, grading, and scope validation
Impact on Project	Ensures alignment with academic standards and improves project quality

2.11.1.3 End Users (Motorcycle Riders)

Attribute	Details
Description	Motorcycle riders in Pakistan, especially urban areas like Lahore
Role	Primary users and beneficiaries of the HelmX system
Responsibilities	Participating in testing, providing feedback, validating usability
Involvement Level	Medium – mainly during testing and feedback phases
Key Interests	Safety, convenience, affordability, smart features like crash detection

Attribute	Details
	and navigation
Influence Level	Medium – feedback influences design and usability improvements
Impact on Project	Shapes user experience and feature refinement

2.11.1.4 University Administration

Attribute	Details
Organization	FCIT, Department of Computer Science, University of the Punjab
Role	Institutional authority overseeing and evaluating the project
Responsibilities	Project approval, providing resources, final evaluation
Involvement Level	Low – limited to administrative and evaluation stages
Key Interests	Academic standards, innovation, institutional reputation
Influence Level	Medium – controls evaluation criteria and approvals
Impact on Project	Determines academic success and compliance

2.11.2 Secondary Stakeholders

2.11.2.1 Emergency Response Services

Attribute	Details
Description	Ambulance services, hospitals, and police departments
Role	Recipients of emergency alerts generated by the system
Responsibilities	Responding to crash alerts
Involvement Level	Low – passive involvement
Key Interests	Accurate and timely alerts, reduced response time
Influence Level	Low – minor input on alert formats
Impact on Project	Improves real-world usefulness of crash detection feature

2.11.2.2 Transportation Safety Authorities

Attribute	Details
Description	NHMP, Provincial Traffic Police
Role	Potential regulators or adopters
Responsibilities	Policy making and traffic safety enforcement
Involvement Level	Low – future potential involvement
Key Interests	Accident reduction, safety improvements, data insights
Influence Level	Low to Medium – may impose regulations in future
Impact on Project	Can influence scalability and legal compliance

2.11.2.3 Hardware and Component Suppliers

Attribute	Details
Description	Vendors of sensors, cameras, batteries, GPS modules
Role	Provide hardware components
Responsibilities	Supplying reliable components
Involvement Level	Low – transactional relationship
Key Interests	Sales and potential bulk demand
Influence Level	Low – affects design constraints
Impact on Project	Impacts hardware reliability and feasibility

2.11.2.4 Cloud Service Providers

Attribute	Details
Organization	AWS, MongoDB Atlas
Role	Provide cloud infrastructure and data services
Responsibilities	Hosting, storage, IoT connectivity
Involvement Level	Low – service-based interaction
Key Interests	Platform usage and scalability
Influence Level	Low – pricing and services affect architecture
Impact on Project	Enables backend functionality and scalability

2.11.2.5 Academic Research Community

Attribute	Details
Description	Researchers in AI, IoT, and intelligent transportation systems
Role	Consumers of research output
Responsibilities	Reviewing and referencing research
Involvement Level	Low – indirect involvement
Key Interests	Innovation, methodology, technical contribution
Influence Level	Low – affects research standards
Impact on Project	Encourages high-quality documentation and innovation

2.11.2.6 Technology Enthusiasts and Early Adopters

Attribute	Details
Description	Individuals interested in smart wearables and IoT
Role	Potential early users and testers
Responsibilities	Providing feedback in pilot stages
Involvement Level	Low – optional participation
Key Interests	Innovation, early access, advanced features

Attribute	Details
Influence Level	Low – may guide future enhancements
Impact on Project	Supports future scalability and adoption

2.12 Requirements Elicitation

The requirements elicitation for HelmX was conducted through a systematic approach to ensure comprehensive identification of user needs, system functionalities, and technical constraints. The primary techniques employed include:

Literature Review and Document Analysis: Extensive review of academic papers, existing smart helmet systems, and motorcycle safety research to identify industry best practices, common safety challenges, and proven technological solutions. Analysis of related work from Kim et al. (2025), Sentiance (2023), Chigee (2025), and other relevant studies informed the feature set and technical approach.

Stakeholder Consultation: Direct discussions with the project supervisor (Dr. Tayyaba Tariq) to align project objectives with academic requirements and research goals. Informal interviews with motorcycle riders in Lahore to understand real-world safety concerns, navigation challenges, and user preferences for helmet-based technology.

Brainstorming Sessions: Collaborative team meetings to identify innovative features, prioritize functionalities based on safety impact and technical feasibility, and define the scope of core versus optional features.

Market Analysis: Examination of commercial products (BMW smart helmets, Skullly AR helmets, Rexing MTC1) to identify gaps in affordability, feature integration, and market suitability for developing nations.

Technical Feasibility Assessment: Evaluation of available hardware components, AI algorithms, cloud services, and development tools to ensure requirements align with technical capabilities and project constraints.

This comprehensive elicitation process resulted in well-defined functional and non-functional requirements that address stakeholder needs while remaining achievable within the project timeline and resource constraints.

2.12.1 Functional Requirements

Functional requirements define the specific behaviors, features, and operations that the HelmX system must perform to meet user needs and project objectives.

2.12.1.1 FR1: Crash Detection System

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR1.1	Continuously monitor accelerometer and gyroscope data from IMU sensor at $\geq 60\text{Hz}$ to detect sudden impacts	IMU sensor data (acceleration, rotation)	Real-time motion stream	Sensors calibrated and active	Continuous data stream maintained	High
FR1.2	Use trained ML classifier to detect crash patterns with $\geq 85\%$ accuracy	Sensor data, trained model	Crash detection signal	Model trained and deployed	Crash classified correctly	High
FR1.3	Capture GPS coordinates with $\leq 10\text{m}$ accuracy upon crash detection	GPS module data	Latitude, Longitude	GPS module active	Accurate location recorded	High
FR1.4	Send emergency alert with location, rider info, timestamp within 10 seconds	Crash signal, GPS data, user profile	Alert notification (SMS/App)	Network connectivity available	Alert successfully delivered	Critical
FR1.5	Log crash data with sensor readings to cloud for analysis	Sensor + crash data	Stored logs in database	Cloud connection active	Data available for analysis	Medium

2.12.1.2 FR2: Drowsiness Detection System

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR2.1	Capture continuous video of rider face at ≥ 15 FPS	Camera feed	Video frames	Camera operational	Continuous video stream	High
FR2.2	Detect eye closure using OpenCV/CNN in real-time	Video frames	Eye state (open/closed)	Model loaded	Eye status detected	High
FR2.3	Trigger alert if eyes closed > 2 seconds	Eye state data	Audio alert	Detection active	Alert generated	Critical
FR2.4	Provide warnings via audio or display	Detection output	Warning signal	Alert system active	User notified	High
FR2.5	Allow user to configure sensitivity via app	User input	Updated settings	App connected	Preferences applied	Medium

2.12.1.3 FR3: Smart Navigation System

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR3.1	Integrate Google Maps API/ OpenStreetMap for real-time	GPS + API data	Route + voice guidance	Internet available	Navigation active	High

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
	navigation					
FR3.2	Optimize routes based on traffic	Traffic data	Optimized route	API connected	Faster route selected	High
FR3.3	Provide voice navigation via helmet speakers	Navigation data	Audio instructions	Speakers functional	Rider guided hands-free	High
FR3.4	Accept destination via voice/app input	Voice/text input	Destination set	Input system active	Route generated	Medium

2.12.1.4 FR4: Dual Camera System

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR4.1	Record video from front & rear cameras in $\geq 720p$	Camera feeds	Video recordings	Cameras active	Footage stored	High
FR4.2	Capture images via voice command	Voice input	Image capture	Voice system active	Image saved	Medium
FR4.3	Compress video for storage optimization	Raw video	Compressed video	Processing unit active	Storage optimized	Medium
FR4.4	Upload footage to cloud storage	Video files	Cloud backup	Internet available	Data stored remotely	High

2.12.1.5 FR5: Voice Control and Entertainment System

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR5.1	Enable voice recognition for system control	Voice commands	Executed actions	NLP system active	Commands processed	High
FR5.2	Connect with smartphone via Bluetooth	Bluetooth signal	Media/call access	Device paired	Connectivity established	High
FR5.3	Support online & offline voice commands	Voice input	Command execution	System configured	Works in both modes	Medium

2.12.1.6 FR6: Safety and Environment Monitoring System

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR6.1	Measure air quality (PM2.5, PM10) every 30 sec	Sensor data	AQ readings	Sensors active	Data updated	High
FR6.2	Monitor ambient temperature	Temp sensor	Temperature data	Sensor active	Data displayed	Medium
FR6.3	Log environmental data for analysis	Sensor logs	Stored data	Storage active	Data available	Medium

2.12.1.7 FR7: Mobile and Web Applications

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR7.1	Display real-time telemetry (speed, location)	Sensor data	Dashboard display	App connected	Data visible	High
FR7.2	Provide ride analytics dashboard	Ride data	Analytics reports	Data stored	Insights generated	Medium
FR7.3	Allow system configuration via app	User input	Updated settings	User logged in	Changes applied	High
FR7.4	Provide secure authentication	User credentials	Login access	Backend active	Secure access granted	Critical

2.12.1.8 FR8: Data Management

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR8.1	Store data in MongoDB with indexing	User/system data	Database records	DB configured	Efficient retrieval	High
FR8.2	Synchronize device, cloud, and apps	System data	Synced state	All systems active	Data consistency	High
FR8.3	Provide API endpoints for apps	API requests	Data responses	Backend running	App functionality enabled	High

ID	Requirement Description	Inputs	Outputs	Preconditions	Postconditions	Priority
FR8.4	Maintain backups and recovery mechanisms	System data	Backup storage	Backup system active	Data recoverable	Critical

2.12.2 Non-Functional Requirements

Non-functional requirements specify the quality attributes, performance characteristics, and constraints that the HelmX system must satisfy.

2.12.2.1 NFR1: Performance Requirements

ID	Requirement	Description	Metric / Constraint	Priority
NFR1.1	Crash detection processing time	System must quickly analyze IMU data to detect crashes in real-time	≤ 300 ms	Critical
NFR1.2	Drowsiness detection latency	Video frames must be processed with minimal delay for timely alerts	≤ 500 ms	Critical
NFR1.3	Emergency alert transmission	Alerts must be sent immediately after crash detection	≤ 10 seconds	Critical
NFR1.4	Application response time	Mobile app should quickly load analytics and ride data	≤ 2 seconds (4G)	High
NFR1.5	Battery performance	System should sustain continuous operation during rides	2–3 hours active usage	High

2.12.2.2 NFR2: Accuracy and Reliability

ID	Requirement	Description	Metric / Constraint	Priority
NFR2.1	Crash detection accuracy	System must reliably detect crashes while minimizing false alarms	$\geq 85\%$, $<10\%$ false positives	Critical
NFR2.2	Drowsiness detection accuracy	Accurate detection across lighting and environmental variations	$\geq 85\%$ accuracy	Critical
NFR2.3	GPS precision	Location data must be sufficiently accurate for emergency response	≤ 10 meters	High
NFR2.4	System availability	System should remain operational during active rides	$\geq 99\%$ uptime	High
NFR2.5	Voice recognition reliability	Commands should be accurately interpreted under normal noise	$\geq 90\%$ accuracy	Medium

2.12.2.3 NFR3: Usability Requirements

ID	Requirement	Description	Metric / Constraint	Priority
NFR3.1	System usability	Overall ease of use measured through user feedback	SUS Score ≥ 70	High
NFR3.2	Voice interaction	Commands should be simple and natural for riders	No training required	High
NFR3.3	User interface design	App UI must be clean, intuitive, and consistent	Standard UI/UX guidelines	High

ID	Requirement	Description	Metric / Constraint	Priority
NFR3.4	Alert clarity	Alerts must be clearly noticeable during riding	Audio/visual/haptic feedback	Critical
NFR3.5	Setup efficiency	New users should quickly configure the system	≤ 10 minutes setup time	Medium

2.12.2.4 NFR4: Security and Privacy

ID	Requirement	Description	Metric / Constraint	Priority
NFR4.1	Secure data transmission	All communication must be protected against interception	TLS encryption	Critical
NFR4.2	Authentication security	User accounts must be securely managed	Hashed passwords + token sessions	Critical
NFR4.3	Data access control	Only authorized users can access personal data	Role-based/authenticated access	High
NFR4.4	Privacy compliance	Users must be informed and consent to data usage	Consent-based data collection	High

2.12.2.5 NFR5: Scalability and Maintainability

ID	Requirement	Description	Metric / Constraint	Priority
NFR5.1	System scalability	Backend should handle increasing number of users	≥ 50 concurrent users	Medium
NFR5.2	Modular architecture	Components should be independently updatable	Loosely coupled modules	High

ID	Requirement	Description	Metric / Constraint	Priority
NFR5.3	Code maintainability	Codebase should be easy to understand and modify	Standard coding practices + documentation	High
NFR5.4	Error handling & logging	System should record issues for debugging	Centralized logging system	High

2.12.2.6 NFR6: Compatibility and Interoperability

ID	Requirement	Description	Metric / Constraint	Priority
NFR6.1	Android compatibility	App should run on a wide range of devices	Android 8.0+	High
NFR6.2	Web compatibility	Dashboard must work across major browsers	Chrome, Firefox, Safari, Edge	High
NFR6.3	Bluetooth integration	Seamless connection with smartphones	Standard Bluetooth protocols	High
NFR6.4	Hardware compatibility	System should fit standard helmets	Standard mounting mechanisms	Medium
NFR6.5	API interoperability	Support integration with external services	REST/API-based (e.g., Maps)	High

2.12.2.7 NFR7: Environmental and Physical Constraints

ID	Requirement	Description	Metric / Constraint	Priority
NFR7.1	Operating temperature	System must function in varied climates	-10°C to 50°C	High

ID	Requirement	Description	Metric / Constraint	Priority
NFR7.2	Weather resistance	Components must tolerate rain and humidity	Water-resistant design	High
NFR7.3	Weight limitation	Added components should not affect comfort	≤ 300 grams	High
NFR7.4	Charging convenience	Easy and standard charging options	USB-C / Micro-USB	Medium

2.12.2.8 NFR8: Cost and Resource Constraints

ID	Requirement	Description	Metric / Constraint	Priority
NFR8.1	Hardware cost control	Prototype must stay within budget limits	Within allocated budget	High
NFR8.2	Cloud cost optimization	Usage should remain cost-efficient during development	AWS free tier utilization	Medium
NFR8.3	Software cost efficiency	Prefer free and open-source tools	Open-source frameworks	High
NFR8.4	Project timeline	Development must align with academic schedule	8–9 months duration	Critical

2.12.3 Requirements Traceability Matrix

The Requirements Traceability Matrix (RTM) maps each requirement to its source, related system components, testing approach, and priority level to ensure comprehensive coverage and accountability

Req ID	Requirement Description	Source	System Component	Priority	Test Method	Status
FR1.1	Continuous IMU monitoring at 60Hz	Stakeholder, Literature	Hardware Layer	High	Unit Test	Done
FR1.2	Crash detection ML classifier (85% accuracy)	Safety Research	Edge AI Layer	High	Integration Test	Done
FR1.3	GPS coordinate capture (10m accuracy)	Safety Requirements	Hardware Layer	High	System Test	Done
FR1.4	Emergency alert transmission (<10s)	Safety Requirements	Cloud Layer	High	Integration Test	Done
FR1.5	Crash event logging	System Requirements	Cloud Layer	Medium	System Test	Pending
FR2.1	Continuous video capture (15fps)	Safety Research	Hardware Layer	High	Unit Test	Done
FR2.2	Eye closure detection	Literature Review	Edge AI Layer	High	Integration Test	Done
FR2.3	Audio alerts for drowsiness	User Needs	Hardware Layer	High	User Acceptance	Done
FR2.4	Configurable sensitivity settings	User Feedback	Application Layer	Low	System Test	Pending
FR3.1	Google Maps/OpenStreetMap integration with voice guidance	User Needs	Application Layer	High	Integration Test	Done
FR3.2	Real-time traffic-based routing	User Convenience	Cloud Layer	Medium	System Test	Done
FR3.3	Voice-based navigation	User Needs	Hardware Layer	High	User Acceptance	Done

	instructions					
FR3.4	Voice/app destination input	User Convenience	Application Layer	Medium	Integration Test	Done
FR4.1	Dual camera recording (720p minimum)	Evidence Collection	Hardware Layer	High	Unit Test	Done
FR4.2	Voice-command image capture	User Convenience	Hardware/Edge AI	Medium	Integration Test	Done
FR4.3	Video compression for storage	Technical Constraint	Edge AI Layer	Medium	Performance Test	Done
FR4.4	Cloud video upload	Evidence Backup	Cloud Layer	Medium	Integration Test	Pending
FR4.5	Timestamp and GPS tagging	Evidence Integrity	Edge AI Layer	High	System Test	Pending
FR5.1	Voice recognition for commands	User Convenience	Edge AI Layer	High	User Acceptance	Pending
FR5.2	Bluetooth media integration	User Needs	Hardware Layer	Medium	Integration Test	Done
FR5.3	Online/offline command support	Reliability	Edge AI Layer	Medium	System Test	Done
FR5.4	Voice feedback for commands	User Experience	Hardware Layer	Low	User Acceptance	Pending
FR6.1	Air quality monitoring (PM2.5, PM10)	Health Awareness	Hardware Layer	Medium	Unit Test	Done
FR6.2	Temperature monitoring	Environmental Data	Hardware Layer	Low	Unit Test	Done
FR6.3	Environmental data logging	Analytics	Cloud Layer	Low	System Test	Pending
FR7.1	Real-time telemetry display	User Monitoring	Application Layer	High	User Acceptance	Pending
FR7.2	Comprehensive ride analytics	User Insights	Application Layer	Medium	System Test	Done
FR7.3	System configuration interface	User Control	Application Layer	High	User Acceptance	Done
FR7.	Historical data	User Insights	Application	Medium	System	Done

4	visualization		Layer	m	Test	
FR7.5	User authentication and login	Security	Application Layer	High	Security Test	Done
FR8.1	Real-time cloud data transmission	System Integration	Cloud Layer	High	Integration Test	Pending
FR8.2	MongoDB data storage with indexing	Data Management	Cloud Layer	High	Performance Test	Done
FR8.3	Device-cloud-app synchronization	System Integration	All Layers	High	Integration Test	Pending
FR8.4	RESTful API endpoints	System Integration	Cloud Layer	High	Integration Test	Done
FR8.5	Data backup and recovery	Reliability	Cloud Layer	Medium	System Test	Pending
NFR 1.1	Crash detection latency <300ms	Performance Goal	Edge AI Layer	High	Performance Test	Done
NFR 1.2	Drowsiness detection latency <500ms	Performance Goal	Edge AI Layer	High	Performance Test	Done
NFR 1.3	Alert transmission <10s	Performance Goal	Cloud Layer	High	Performance Test	Done
NFR 1.4	App data load time <3s	User Experience	Application Layer	Medium	Performance Test	Done
NFR 1.5	Battery life 2-3 hours	Hardware Constraint	Hardware Layer	High	System Test	Done
NFR 2.1	Crash detection 85% accuracy, <10% FP	Quality Goal	Edge AI Layer	High	Validation Test	Done
NFR 2.2	Drowsiness detection 85% accuracy	Quality Goal	Edge AI Layer	High	Validation Test	Done
NFR 2.3	GPS accuracy within 10m	Quality Goal	Hardware Layer	High	System Test	Done
NFR 2.4	System uptime 99%	Reliability Goal	All Layers	High	System Test	Done
NFR 2.5	Voice recognition >85% accuracy	Quality Goal	Edge AI Layer	Medium	Validation Test	Pending
NFR	SUS score >70	Usability Goal	Application	High	User	Done

3.1			Layer		Acceptance	
NFR 4.1	TLS encryption for data transmission	Security Standard	Cloud Layer	High	Security Test	Pending
NFR 4.2	Secure authentication mechanisms	Security Standard	Application Layer	High	Security Test	Done
NFR 5.1	Support 50 concurrent users	Scalability Goal	Cloud Layer	Medium	Load Test	Done
NFR 6.1	Android 8.0+ compatibility	Platform Support	Application Layer	High	Compatibility Test	Done
NFR 6.2	Cross-browser web compatibility	Platform Support	Application Layer	Medium	Compatibility Test	Done
NFR 7.1	Operating temperature - 10°C to 50°C	Environmental Spec	Hardware Layer	Medium	Environmental Test	Done
NFR 7.2	Weather-resistant components	Environmental Spec	Hardware Layer	Medium	Environmental Test	Done
NFR 8.1	Stay within project budget	Resource Constraint	All Layers	High	Budget Review	Pending
NFR 8.4	Complete within 8-9 months	Time Constraint	All Layers	High	Schedule Review	Done

2.13 Use Case Description

This section presents key use case descriptions for the HelmX smart helmet system, illustrating how different actors interact with the system to accomplish specific goals. Each use case follows a structured format including actors, preconditions, main flow, alternate flows, and postconditions.

2.13.1 Use Case 1: Crash Detection and Emergency Alert

Attribute	Details
Use Case ID	UC-01
Use Case Name	Automatic Crash Detection and Emergency Notification

Attribute	Details
Primary Actor	Motorcycle Rider
Secondary Actors	Emergency Contacts, Emergency Services
Priority	Critical

Description: The system automatically detects when the rider has been involved in a crash and sends emergency alerts with GPS location to designated contacts.

Preconditions:

- Helmet is powered on and authenticated
- IMU sensors are calibrated and operational
- GPS signal acquired with location accuracy within 10 meters
- Emergency contacts configured in mobile application
- Network connectivity available (cellular or WiFi)

Main Flow:

1. Rider wears authenticated helmet and begins motorcycle journey
2. IMU sensors continuously monitor acceleration and gyroscope data at 60 Hz
3. Edge AI model processes IMU data in real-time using sliding window approach
4. System detects impact event with reconstruction error exceeding threshold
5. System captures GPS coordinates immediately upon crash detection
6. System automatically sends SMS alerts to all configured emergency contacts
7. SMS contains crash timestamp, GPS coordinates (latitude/longitude), and Google Maps link
8. Mobile application displays crash alert notification to rider (if conscious)
9. System logs crash event with full sensor data to database for analysis

Alternate Flow 1: False Positive Detection

- 4a. Rider experiences harsh braking or speed bump that triggers detection
- 4b. System displays countdown timer (10 seconds) allowing cancellation
- 4c. Rider issues voice command "Cancel alert"
- 4d. Alert cancelled, no SMS sent, event logged as false positive
- 4e. Use case ends

Alternate Flow 2: Network Connectivity Lost

- 6a. No network available at crash location
- 6b. System queues alert in local storage with retry mechanism
- 6c. Continuously attempts to send alert every 30 seconds

- 6d. Alert sent when connectivity restored
- 6e. Resume at step 7

Postconditions:

- Emergency contacts receive SMS notification within 10 seconds of crash
- Crash event logged in database with complete sensor data
- GPS coordinates available for emergency responders
- Rider profile updated with crash incident record

2.13.2 Use Case 2: Drowsiness Detection and Alert

Attribute	Details
Use Case ID	UC-02
Use Case Name	Drowsiness Detection and Alert System
Primary Actor	Motorcycle Rider
Secondary Actors	—
Priority	Critical

Description: The system continuously monitors the rider's facial features to detect signs of drowsiness and provides immediate alerts to prevent accidents.

Preconditions:

- Helmet powered on and camera operational
- Rider face visible to front-facing camera
- Drowsiness detection model loaded on edge device
- Audio and visual alert systems functional
- Adequate lighting for facial feature detection (or IR LED active at night)

Main Flow:

1. Rider begins journey with helmet camera active
2. Camera captures video feed at 15 frames per second
3. MediaPipe detects facial landmarks in each frame
4. System calculates Eye Aspect Ratio (EAR) from eye landmarks
5. MobileNetV2 model classifies current state as "Alert" or "Drowsy"

6. System continues monitoring without alerts while rider is alert
7. Rider begins showing drowsiness indicators (eyes closing)
8. EAR falls below 0.25 threshold for sustained 2 seconds
9. System triggers multi-modal alert: audio warning, LED flashing, vibration
10. Alert continues until rider demonstrates alert state for 10 consecutive seconds
11. Drowsiness event logged with timestamp and severity level
12. Mobile app receives notification of drowsiness incident

Alternate Flow 1: Temporary Eye Closure (Blink)

- 9a. EAR drops below threshold but recovers within 0.5 seconds (normal blink)
- 9b. No alert triggered, monitoring continues
- 9c. Resume at step 7

Alternate Flow 2: Camera Obstruction

- 3a. Camera view blocked by visor condensation, dirt, or improper positioning
- 3b. Face detection fails for 10 consecutive frames
- 3c. Audio prompt: "Drowsiness detection unavailable"
- 3d. Rider adjusts helmet or clears obstruction
- 3e. Resume at step 2

Postconditions:

- Rider alerted to drowsiness state before severe impairment
- Drowsiness incidents recorded in ride history
- System ready to detect subsequent drowsiness episodes
- Analytics data available for rider to review drowsiness patterns

2.13.3 Use Case 3: Voice-Controlled Navigation

Attribute	Details
Use Case ID	UC-03
Use Case Name	Smart Navigation Assistance
Primary Actor	Motorcycle Rider
Secondary Actors	Google Maps API/ OpenStreetMap
Priority	High

Description: Rider uses voice commands to initiate navigation to a destination and receives turn-by-turn guidance without manual interaction with devices.

Preconditions:

- Helmet paired with smartphone via Bluetooth
- Google Maps installed on paired smartphone
- Voice recognition system active
- GPS signal available
- Audio output functional (helmet speakers)

Main Flow:

1. Rider wears authenticated helmet and starts motorcycle
2. Rider issues voice command: "Navigate to [destination name]"
3. Voice recognition system processes command and extracts destination
4. System sends navigation request to paired smartphone
5. Google Maps on smartphone calculates optimal route
6. Navigation session begins on smartphone
7. Helmet's NotificationListenerService captures Google Maps notifications
8. System parses turn-by-turn instructions and distance information
9. As rider approaches turns, system provides voice guidance: "In 500 meters, turn left"
10. Navigation continues until destination reached
11. System announces: "You have arrived at your destination"
12. Navigation session ends, ride statistics updated

Alternate Flow 1: Voice Command Not Recognized

- 3a. Background noise prevents accurate command recognition
- 3b. System responds: "Sorry, I didn't understand. Please repeat."
- 3c. Rider repeats command with clearer pronunciation
- 3d. Resume at step 3

Alternate Flow 2: Route Deviation

- 8a. Rider misses a turn or takes wrong route
- 8b. Google Maps recalculates route automatically
- 8c. System receives updated navigation instructions
- 8d. Audio prompt: "Recalculating route"
- 8e. Resume at step 9 with new instructions

Alternate Flow 3: Manual Navigation Cancellation

- 6a. Rider decides to cancel navigation mid-journey
- 6b. Rider issues voice command: "Cancel navigation"
- 6c. System stops processing navigation notifications
- 6d. Audio confirmation: "Navigation cancelled"
- 6e. Use case ends

Postconditions:

- Rider reaches intended destination safely
- Navigation statistics logged (route taken, duration, distance)
- System ready for next navigation request
- No manual device interaction required during ride

2.13.4 Use Case 4: Ride Analytics Review

Attribute	Details
Use Case ID	UC-04
Use Case Name	Post-Ride Data Analysis and Insights
Primary Actor	Motorcycle Rider
Secondary Actors	Cloud Storage Services
Priority	High

Description: Rider reviews completed ride statistics, safety events, and environmental data through the mobile application or web dashboard.

Preconditions:

- Rider has completed at least one ride with HelmX system
- Ride data synced to cloud database
- Rider logged into mobile app
- Network connectivity available

Main Flow:

1. Rider opens HelmX mobile application after completing journey
2. App displays home screen with recent ride summary card
3. Rider taps on most recent ride to view detailed analytics
4. App retrieves ride data from Firebase database
5. Dashboard displays comprehensive ride information:
 - Route map with GPS trace
 - Total distance traveled (e.g., 24.5 km)
 - Ride duration (e.g., 45 minutes)
 - Average speed (e.g., 32 km/h)
 - Maximum speed (e.g., 78 km/h)
6. Safety events section shows:

- Number of drowsiness alerts (e.g., 2 events)
- Harsh braking incidents (if any)
- Speed limit violations (if applicable)
- 7. Environmental data section displays:
 - Average air quality index (e.g., AQI 68 - Moderate)
 - Temperature range during ride (e.g., 28-32°C)
- 8. Rider reviews data and identifies patterns
- 9. Rider navigates to "All Rides" to compare historical trends
- 10. Rider identifies improvement areas (e.g., drowsiness during evening rides)
- 11. Rider adjusts riding schedule

Alternate Flow 1: No Network Connectivity

- 4a. No internet connection available when opening app
- 4b. App displays cached data from last sync
- 4c. Banner notification: "Viewing offline data. Connect to sync latest rides."
- 4d. Rider connects to WiFi later
- 4e. App automatically syncs new ride data in background
- 4f. Notification: "Ride data updated"

Postconditions:

- Rider gains insights into riding behavior and safety patterns
 - Historical ride data available for trend analysis
 - Rider can make informed decisions to improve safety
 - Data available for sharing with third parties (insurance, fleet management)
-

Chapter 3. System Design

3.1 Use Case Design

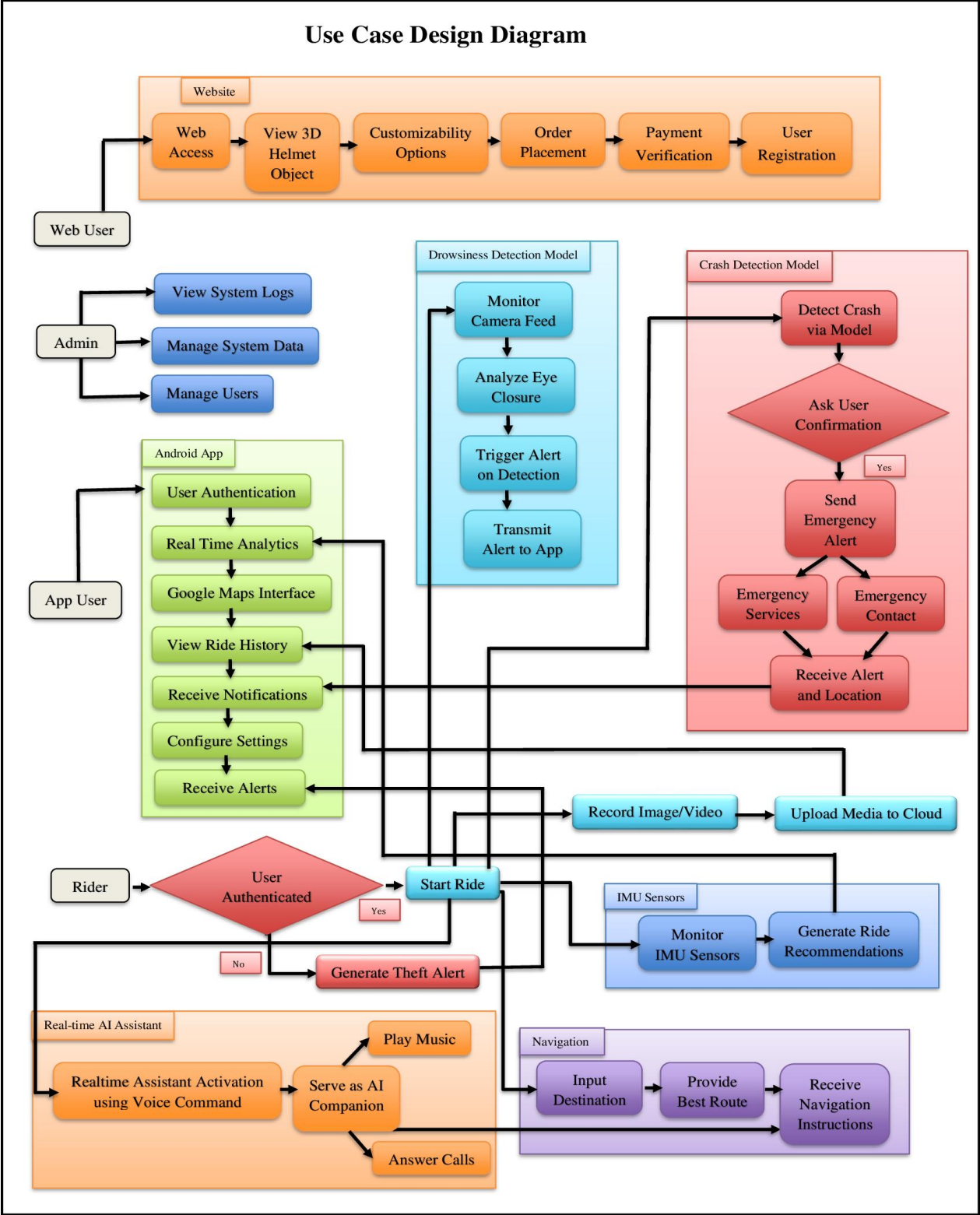


Figure 1 Use Case Design Diagram

3.2 System Architecture Diagram

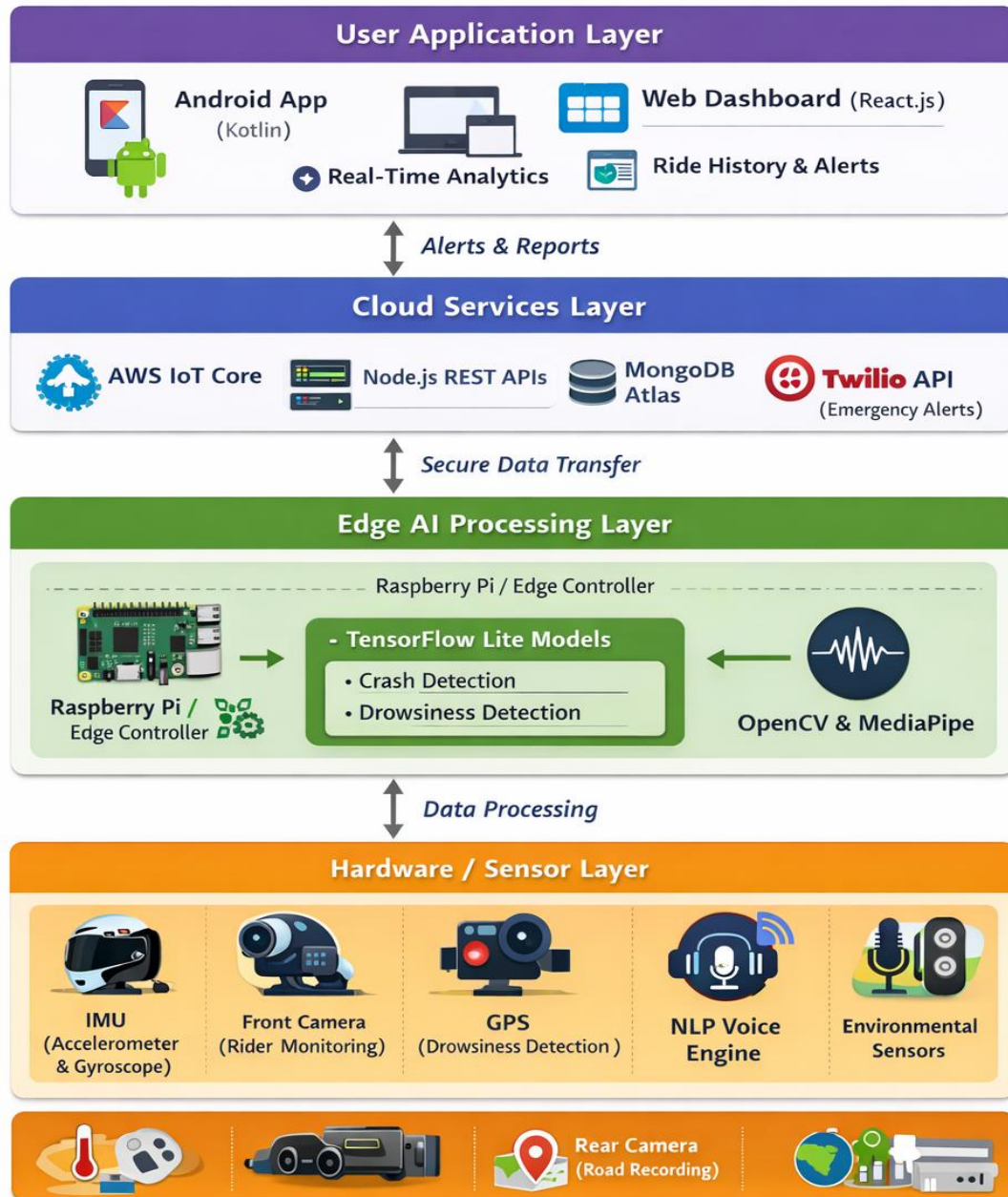


Figure 2 System Architecture Diagram

Chapter 4. System Testing

This chapter outlines the comprehensive testing strategy for the HelmX smart helmet system, covering test case design, unit testing, integration testing, and user acceptance testing. The testing approach ensures that all seven core features meet the defined functional and non-functional requirements, with particular emphasis on safety-critical components such as crash detection and drowsiness monitoring.

4.1 Test Cases Design

The test cases for HelmX are designed following IEEE 829 standard for software test documentation, organized by functional modules and prioritized based on safety criticality and system dependencies. Each test case includes test case ID, description, preconditions, test steps, expected results, actual results, and pass/fail status.

4.1.1 Crash Detection Test Cases

Test Case CD-001: Normal Riding Acceleration Detection

- **Objective:** Verify system does not trigger false crash alerts during normal acceleration and deceleration
- **Preconditions:** Helmet powered on, IMU sensor calibrated, GPS signal acquired
- **Test Steps:**
 1. Mount helmet on test rider
 2. Accelerate motorcycle from 0 to 60 km/h over 10 seconds
 3. Decelerate from 60 km/h to 0 over 5 seconds
 4. Repeat for 20 cycles
- **Expected Result:** Zero false positive crash alerts during all normal riding maneuvers
- **Pass Criteria:** False positive rate < 10%

Test Case CD-002: Simulated Frontal Impact Detection

- **Objective:** Verify system detects frontal collision with >90% accuracy
- **Preconditions:** Helmet secured on crash test dummy; emergency contacts configured
- **Test Steps:**
 1. Mount helmet on crash test dummy or actual person at low speed.
 2. Simulate frontal impact at 30 km/h using crash test apparatus
 3. Record time from impact to alert generation
 4. Verify GPS coordinates captured
 5. Verify emergency SMS sent to configured contacts
- **Expected Result:** Crash detected within 200ms, alert sent within 10 seconds, GPS accuracy within 10 meters
- **Pass Criteria:** Detection accuracy >85%, alert latency <5 seconds

Test Case CD-003: Harsh Braking Differentiation

- **Objective:** Verify system distinguishes harsh braking from actual crashes
- **Preconditions:** Helmet operational, baseline IMU data collected
- **Test Steps:**
 1. Perform emergency braking from 80 km/h to full stop
 2. Record IMU sensor data and system response
 3. Repeat for 15 trials with varying deceleration rates
- **Expected Result:** No crash alerts during controlled emergency braking

- **Pass Criteria:** False positive rate during harsh braking <10%

Test Case CD-004: Speed Bump Navigation

- **Objective:** Ensure speed bumps and potholes do not trigger false crash detection
- **Preconditions:** Helmet active, riding at 20-40 km/h
- **Test Steps:**
 1. Navigate over standard speed bumps at various speeds
 2. Traverse rough road surfaces with potholes
 3. Monitor for false positive alerts
- **Expected Result:** No crash alerts during typical road anomalies
- **Pass Criteria:** Zero false positives across 50 speed bump/pothole encounters

Test Case CD-005: Multi-Type Crash Detection

- **Objective:** Validate detection across different crash scenarios
- **Preconditions:** Controlled test environment, safety measures in place
- **Test Steps:**
 1. Simulate lateral impact (side collision)
 2. Simulate rear-end collision
 3. Simulate rollover scenario
 4. Record detection accuracy for each crash type
- **Expected Result:** >85% detection accuracy across all crash types
- **Pass Criteria:** Minimum 85% detection rate for each scenario

4.1.2 Drowsiness Detection Test Cases

Test Case DD-001: Alert State Recognition

- **Objective:** Verify system correctly identifies alert, non-drowsy state
- **Preconditions:** Camera operational, adequate lighting, rider facing forward
- **Test Steps:**
 1. Rider maintains normal alert posture for 10 minutes
 2. Record Eye Aspect Ratio (EAR).
 3. Verify no drowsiness alerts generated
- **Expected Result:** System maintains "alert" classification with no false drowsiness warnings
- **Pass Criteria:** Zero false drowsiness alerts during 10-minute alert riding session

Test Case DD-002: Eye Closure Detection

- **Objective:** Validate detection of prolonged eye closure indicating drowsiness
- **Preconditions:** Rider equipped with helmet, system active
- **Test Steps:**
 1. Rider closes eyes for >2 seconds while stationary (safe conditions)
 2. Record time from closure to alert generation
 3. Verify audio and visual alerts triggered
 4. Repeat 20 times with varying closure durations (2-5 seconds)
- **Expected Result:** Alert triggered within 500ms of exceeding 2-second threshold, >85% detection accuracy
- **Pass Criteria:** Detection accuracy >85%, alert latency <500ms

Test Case DD-003: Variable Lighting Conditions

- **Objective:** Ensure drowsiness detection works across different lighting scenarios
- **Preconditions:** Controlled lighting environment
- **Test Steps:**
 1. Test detection in bright daylight
 2. Test in low light conditions
 3. Test during sunset/sunrise transition periods
 4. Test with IR LED assistance at night
- **Expected Result:** >85% detection accuracy maintained across all lighting conditions
- **Pass Criteria:** Minimum 85% accuracy in each lighting scenario

4.1.3 Smart Navigation Test Cases

Test Case SN-001: Automatic Navigation Launch

- **Objective:** Verify automatic capture of Google Maps navigation session
- **Preconditions:** Smartphone paired via Bluetooth, Google Maps installed
- **Test Steps:**
 1. Start navigation on paired smartphone
 2. Verify helmet receives navigation intent automatically
 3. Confirm no manual helmet configuration required
- **Expected Result:** Helmet automatically detects navigation session within 2 seconds
- **Pass Criteria:** 100% automatic detection across 20 navigation sessions

Test Case SN-002: Turn-by-Turn Instruction Delivery

- **Objective:** Validate timely delivery of navigation instructions
- **Preconditions:** Active navigation route, BLE connection established
- **Test Steps:**
 1. Follow predetermined route with multiple turns
 2. Record time between Google Maps notification and helmet alert
 3. Measure BLE communication latency
 4. Verify audio guidance clarity at different speeds
- **Expected Result:** BLE latency <150ms, audio guidance clear at all speeds
- **Pass Criteria:** Average latency <150ms, 95% instruction clarity rating

Test Case SN-003: GPS Signal Loss Recovery

- **Objective:** Test navigation behavior during GPS signal interruption
- **Preconditions:** Active navigation in area with intermittent GPS
- **Test Steps:**
 1. Navigate through tunnel or urban canyon
 2. Monitor system response to GPS signal loss
 3. Verify last-known position maintained
 4. Confirm automatic recovery when signal restored
- **Expected Result:** System maintains last position, resumes navigation within 5 seconds of signal restoration
- **Pass Criteria:** Smooth recovery in >90% of signal loss events

Test Case SN-004: Battery Consumption During Navigation

- **Objective:** Measure power consumption during continuous navigation
- **Preconditions:** Fully charged battery, 3-hour target operational time
- **Test Steps:**
 1. Activate continuous navigation for 3-hour session
 2. Monitor battery percentage decline
 3. Calculate projected operational duration
- **Expected Result:** Battery consumption allows >3 hours continuous navigation
- **Pass Criteria:** Projected battery life >3 hours at measured consumption rate

4.1.4 Dual Camera System Test Cases

Test Case DC-001: Continuous Recording Quality

- **Objective:** Verify video quality and storage management during continuous recording
- **Preconditions:** Sufficient storage available (64GB minimum), cameras operational
- **Test Steps:**
 1. Enable continuous recording on both front and rear cameras
 2. Record 2-hour riding session
 3. Review footage for clarity, frame rate consistency, timestamp accuracy
 4. Verify automatic loop recording when storage threshold reached
- **Expected Result:** 720p minimum resolution maintained, consistent frame rate, accurate timestamps, automatic loop recording functional
- **Pass Criteria:** Video quality meets 720p standard, no frame drops, loop recording activates at 90% storage

Test Case DC-002: Voice-Command Image Capture

- **Objective:** Test voice-activated still image capture functionality
- **Preconditions:** Voice recognition active, cameras operational
- **Test Steps:**
 1. Issue voice command "Capture image" while riding
 2. Verify image captured from both cameras
 3. Check image quality and GPS tagging
 4. Repeat 15 times at various speeds and noise levels
- **Expected Result:** >90% voice command recognition, high-quality images with GPS tags
- **Pass Criteria:** Command recognition >90%, all images properly tagged

Test Case DC-003: Low-Light Recording Performance

- **Objective:** Assess camera performance in low-light conditions
- **Preconditions:** Night or low-light environment
- **Test Steps:**
 1. Record footage during dusk/night conditions
 2. Evaluate visibility of road markings, vehicles, obstacles
 3. Test IR LED assistance if available
- **Expected Result:** Adequate visibility for evidence documentation
- **Pass Criteria:** License plates readable at 10m distance, road hazards visible

Test Case DC-004: Video Compression Efficiency

- **Objective:** Verify video compression maintains quality while optimizing storage
- **Preconditions:** H.265 codec enabled
- **Test Steps:**
 1. Record identical 1-hour session with H.264 and H.265 codecs
 2. Compare file sizes
 3. Assess quality degradation
- **Expected Result:** H.265 reduces file size by ~50% with minimal quality loss
- **Pass Criteria:** File size reduction >40%, quality remains acceptable for evidence

Test Case DC-005: Cloud Upload Functionality

- **Objective:** Test automatic upload of critical footage to cloud storage
- **Preconditions:** Internet connectivity available, cloud account configured
- **Test Steps:**
 1. Simulate crash detection event
 2. Verify automatic upload of preceding 30 seconds footage
 3. Measure upload time and data integrity
- **Expected Result:** Upload initiated within 10 seconds, complete within 2 minutes on 4G
- **Pass Criteria:** Upload success rate >95%, no data corruption

4.1.5 Voice Control and Entertainment Test Cases

Test Case VC-001: Basic Voice Command Recognition

- **Objective:** Validate recognition of core voice commands
- **Preconditions:** Microphone operational, voice model trained
- **Test Steps:**
 1. Issue commands: "Play music", "Pause", "Next track", "Call [contact]", "Navigate to [location]"
 2. Test at various speeds: stationary, 40 km/h, 80 km/h
 3. Record recognition accuracy
- **Expected Result:** >85% recognition accuracy at all tested speeds
- **Pass Criteria:** Minimum 85% accuracy across all commands and speeds

Test Case VC-002: Multi-Source Audio Management

- **Objective:** Test prioritization between music, navigation, phone calls
- **Preconditions:** Multiple audio sources active simultaneously
- **Test Steps:**
 1. Play music via Bluetooth
 2. Trigger navigation instruction
 3. Receive incoming phone call
 4. Verify appropriate audio mixing/prioritization
- **Expected Result:** Navigation ducking music, call interrupting all other audio
- **Pass Criteria:** Correct priority handling in 100% of scenarios

Test Case VC-003: Offline Voice Command Support

- **Objective:** Ensure critical commands work without internet connectivity
- **Preconditions:** No cellular/WiFi connection, offline voice model loaded

- **Test Steps:**
 1. Disable all network connections
 2. Test basic commands: "Pause", "Volume up", "Capture image"
 3. Verify functionality
- **Expected Result:** Core commands functional offline
- **Pass Criteria:** >90% success rate for offline-supported commands

4.1.6 Safety and Environment Monitoring Test Cases

Test Case EM-001: Temperature Monitoring

- **Objective:** Verify temperature sensor accuracy
- **Preconditions:** Temperature sensor operational, reference thermometer available
- **Test Steps:**
 1. Measure temperature in controlled environments: 0°C, 25°C, 40°C
 2. Compare helmet readings with calibrated thermometer
 3. Calculate measurement error
- **Expected Result:** Accuracy within $\pm 2^{\circ}\text{C}$ across operating range
- **Pass Criteria:** Maximum error $< 2^{\circ}\text{C}$ at all test points

Test Case EM-002: Environmental Alert Generation

- **Objective:** Test alert system for hazardous environmental conditions
- **Preconditions:** Environmental thresholds configured in app
- **Test Steps:**
 1. Simulate high pollution environment (AQI >150)
 2. Verify mobile app notification generated
 3. Check recommendation content (e.g., "Consider indoor parking")
- **Expected Result:** Alert generated within 30 seconds of threshold breach
- **Pass Criteria:** 100% alert generation for threshold violations

4.1.7 Mobile and Web Application Test Cases

Test Case APP-001: Real-Time Telemetry Display

- **Objective:** Verify Android app displays live helmet data accurately
- **Preconditions:** Helmet and phone paired, active riding session
- **Test Steps:**
 1. Monitor app during ride
 2. Verify speed, location, battery percentage update in real-time
 3. Check update frequency (target: 1 second intervals)
- **Expected Result:** All telemetry data updates within 2 seconds of actual change
- **Pass Criteria:** Update latency < 2 seconds, 100% data accuracy

Test Case APP-002: Ride Analytics Visualization

- **Objective:** Test historical ride data presentation
- **Preconditions:** Completed rides stored in database
- **Test Steps:**
 1. Access ride history in mobile app
 2. Select individual ride
 3. Verify display of route map, duration, distance, average speed, safety events

- **Expected Result:** All ride metrics displayed accurately with interactive map
- **Pass Criteria:** Data accuracy 100%, map rendering correct

Test Case APP-003: System Configuration Interface

- **Objective:** Validate user ability to configure helmet settings via app
- **Preconditions:** App connected to helmet
- **Test Steps:**
 1. Modify user profile
 2. Enable/disable specific features
 3. Verify changes sync to helmet
- **Expected Result:** Settings changes applied to helmet within 5 seconds
- **Pass Criteria:** 100% successful configuration updates

Test Case APP-004: Web Dashboard Functionality

- **Objective:** Test web interface for efficient Helmet viewing and buying
- **Preconditions:** User has opened the website and is on main page
- **Test Steps:**
 1. Access web dashboard from browser
 2. View the helmet object and select the configurations
 3. Test payment process and mobile app account creation
 4. Verify user has received the confirmation messages
- **Expected Result:** Responsive interface, accurate data, successful CSV export
- **Pass Criteria:** All features functional, data matches mobile app

Test Case APP-005: Cross-Platform Data Synchronization

- **Objective:** Ensure data consistency between Android app, web dashboard, and helmet
- **Preconditions:** User logged in on multiple devices
- **Test Steps:**
 1. Modify settings on web dashboard
 2. Verify changes reflected in mobile app within 30 seconds
 3. Confirm helmet receives updated configuration
 4. Test reverse sync (helmet → app → web)
- **Expected Result:** Changes propagate across all platforms within 1 minute
- **Pass Criteria:** 100% synchronization success, no data conflicts

4.1.8 Security and Authentication Test Cases

Test Case SEC-001: Voice PIN Authentication

- **Objective:** Test voice-based PIN entry as authentication
- **Preconditions:** Helmet is available and active
- **Test Steps:**
 1. User speaks configured PIN code
 2. System verifies PIN and voice match
 3. Test with background noise
- **Expected Result:** >90% recognition accuracy, helmet unlocks on correct PIN
- **Pass Criteria:** Minimum 90% accuracy, zero false accepts

Test Case SEC-002: Theft Detection Alerts

- **Objective:** Verify alert generation when helmet moved without authorization
- **Preconditions:** Helmet locked, theft detection armed
- **Test Steps:**
 1. Move helmet without authentication
 2. Monitor for mobile app alert
 3. Verify GPS location transmitted
 4. Check alert delivery time
- **Expected Result:** Alert within 10 seconds, accurate GPS coordinates
- **Pass Criteria:** 100% alert generation, location accuracy within 10 meters

Test Case SEC-003: Multi-User Authorization

- **Objective:** Test management of multiple authorized users
- **Preconditions:** Primary user and secondary user enrolled
- **Test Steps:**
 1. Authenticate as primary user
 2. Add secondary user via app
 3. Verify secondary user can unlock helmet
 4. Remove secondary user access
 5. Confirm access revocation
- **Expected Result:** Successful authorization management for multiple users
- **Pass Criteria:** 100% success in add/remove operations, immediate access changes

4.2 Unit / Integration / Acceptance Testing

4.2.1 Unit Testing

Unit testing focuses on individual components and modules in isolation, ensuring each function performs as designed before integration. The HelmX unit testing strategy employs automated test frameworks with code coverage targets of >80% for all critical modules.

Hardware Component Unit Tests:

IMU Sensor Module Tests:

- **Test:** MPU6050 sensor initialization and calibration
- **Method:** Verify sensor returns valid accelerometer and gyroscope readings within expected ranges (-2g to +2g for accelerometer, -250°/s to +250°/s for gyroscope)
- **Coverage:** Initialization routine, data acquisition functions, calibration algorithm
- **Tools:** Arduino Serial Monitor, oscilloscope for signal validation
- **Pass Criteria:** Successful initialization in >99% of power-on cycles, readings stable within ±2% of expected values

Camera Module Tests:

- **Test:** Camera image capture and encoding
- **Method:** Capture test images under various lighting conditions, verify resolution, format, and encoding

- **Coverage:** Camera initialization, frame capture, H.265 encoding
- **Tools:** OpenCV test scripts
- **Pass Criteria:** Successful capture >95% of attempts, encoding time <100ms per frame

GPS Module Tests:

- **Test:** GPS coordinate acquisition accuracy
- **Method:** Compare GPS readings against known reference locations
- **Coverage:** GPS initialization, coordinate extraction
- **Tools:** GPS simulator, known waypoint coordinates
- **Pass Criteria:** Position accuracy within ± 5 meters in open sky conditions

Software Module Unit Tests:

Crash Detection Algorithm Tests:

- **Test:** Autoencoder model inference on sample IMU data
- **Method:** Feed pre-recorded normal and crash IMU sequences to model, verify classifications
- **Coverage:** Data preprocessing, model inference, reconstruction error calculation, threshold comparison
- **Tools:** Python, TensorFlow testing utilities
- **Test Cases:** 1000 normal riding samples, 200 simulated crash samples
- **Pass Criteria:** >95% accuracy on test dataset, inference time <50ms per window

Drowsiness Detection Tests:

- **Test:** Eye Aspect Ratio calculation
- **Method:** Process pre-labeled facial images, verify EAR calculations and classifications
- **Coverage:** Face landmark detection, EAR computation, classification logic
- **Tools:** Python, MediaPipe test images
- **Test Cases:** 500 alert state images, 500 drowsy state images
- **Pass Criteria:** >92% classification accuracy, processing time <100ms per frame

Voice Recognition Tests:

- **Test:** Command parsing and intent extraction
- **Method:** Feed recorded voice commands, verify correct intent identified
- **Coverage:** Audio preprocessing, voice-to-text conversion, command matching
- **Tools:** Speech recognition test dataset, Python
- **Test Cases:** 50 unique commands $\times$ 10 speakers = 500 samples
- **Pass Criteria:** >85% command recognition accuracy across all speakers

GPS Navigation Parser Tests:

- **Test:** Google Maps notification parsing
- **Method:** Simulate navigation notifications, verify direction and distance extraction
- **Coverage:** Notification capture, JSON parsing, direction/distance extraction
- **Tools:** Android JUnit tests, mock notification generator
- **Test Cases:** 100 varied navigation instructions

- **Pass Criteria:** 100% successful parsing, zero data extraction errors

API Endpoint Tests:

- **Test:** Backend REST API functionality
- **Method:** Send HTTP requests to each endpoint, verify responses
- **Coverage:** Authentication, ride data upload, configuration retrieval, emergency alert triggering
- **Tools:** Postman automated tests, Node.js Mocha framework
- **Test Cases:** 50 endpoint variations covering success and error conditions
- **Pass Criteria:** All endpoints return correct HTTP status codes and JSON payloads

4.2.2 Integration Testing

Integration testing validates the interaction between multiple components, ensuring data flows correctly through the system architecture from hardware sensors to cloud services to user applications.

Level 1 Integration: Hardware-to-Edge Processing

IMU-to-Crash Detection Integration:

- **Objective:** Verify IMU data successfully flows to crash detection ML model
- **Test Setup:** Physical helmet with mounted IMU, edge computing unit running inference
- **Test Procedure:**
 1. Generate controlled IMU signals via physical movements
 2. Monitor data pipeline from sensor read to model prediction
 3. Verify prediction results logged correctly
- **Validation:** End-to-end latency <300ms, prediction accuracy matches unit test baseline
- **Issues to Detect:** Data serialization errors, timing synchronization problems, buffer overflows

Camera-to-Drowsiness Detection Integration:

- **Objective:** Validate camera feed processing pipeline
- **Test Setup:** Helmet camera connected to Raspberry Pi running MediaPipe and MobileNetV2
- **Test Procedure:**
 1. Stream live video from camera
 2. Monitor face detection, landmark extraction, EAR calculation
 3. Verify drowsiness classification output
- **Validation:** Pipeline latency <500ms, classification accuracy consistent with isolated model tests
- **Issues to Detect:** Frame drops, synchronization lag, memory leaks during continuous operation

Level 2 Integration: Edge-to-Cloud Communication

Crash Detection to Emergency Alert Integration:

- **Objective:** Test complete crash-to-notification workflow

- **Test Setup:** Full helmet system, test smartphone with configured emergency contacts, cloud backend operational
- **Test Procedure:**
 1. Trigger simulated crash detection on edge device
 2. Verify crash data transmitted to cloud.
 3. Confirm system invoked to send SMS
 4. Verify SMS received by emergency contacts with GPS coordinates
- **Validation:** End-to-end time from crash detection to SMS delivery <10 seconds
- **Issues to Detect:** Network connectivity failures, API authentication errors, message formatting issues, GPS coordinate transmission errors

Sensor Data to Database Integration:

- **Objective:** Ensure continuous sensor data logging to MongoDB
- **Test Setup:** All helmet sensors active, cloud backend recording data
- **Test Procedure:**
 1. Conduct 1-hour simulated ride with all sensors generating data
 2. Verify data written to MongoDB in real-time
 3. Query database for completeness and accuracy
- **Validation:** >99% data samples successfully stored, query responses <100ms
- **Issues to Detect:** Database connection drops, data loss during transmission, schema mismatches, indexing problems

Level 3 Integration: Cloud-to-Application

Backend API to Mobile App Integration:

- **Objective:** Validate mobile app data retrieval and display
- **Test Setup:** Android app installed on test device, user account with ride history
- **Test Procedure:**
 1. Launch mobile app and authenticate
 2. Request ride history via API
 3. Verify data displayed correctly in app UI
 4. Test real-time telemetry updates during active ride
- **Validation:** API response time <2 seconds, data accuracy 100%, UI updates smoothly
- **Issues to Detect:** Authentication token expiration, JSON parsing errors, UI rendering bugs, data synchronization delays

Level 4 Integration: End-to-End System Testing

Complete Ride Workflow Integration:

- **Objective:** Validate entire system from helmet power-on through ride completion and data analysis
- **Test Setup:** Full HelmX system with all components operational
- **Test Procedure:**
 1. Power on helmet, authenticate user via face recognition
 2. Start navigation to destination via voice command
 3. Ride for 30 minutes with all sensors active
 4. Simulate one drowsiness event (eye closure)
 5. Complete ride, power off helmet
 6. Access ride data via mobile app and web dashboard

- **Validation:** All features functional throughout ride, data accurately captured and displayed, battery consumption within targets
- **Issues to Detect:** Component interaction failures, race conditions, cumulative errors, resource exhaustion

Multi-User Scenario Integration(Optional):

- **Objective:** Test system with multiple concurrent users
- **Test Setup:** 5 helmets with different users simultaneously active
- **Test Procedure:**
 1. All users authenticate and begin rides
 2. Generate mix of events: 1 crash alert, 2 drowsiness alerts, navigation usage
 3. Verify cloud handles concurrent connections
 4. Confirm each user's data isolated correctly
- **Validation:** No cross-user data leakage, all events processed correctly, system remains responsive
- **Issues to Detect:** Database race conditions, API rate limiting issues, authentication conflicts

4.3 User Acceptance Testing

User Acceptance Testing (UAT) ensures that **HelmX meets stakeholder requirements and delivers real value to riders**. Testing is conducted with real motorcycle riders in authentic riding conditions to validate usability, reliability, and safety.

4.3.1 Test Participants

- **Sample Size:** 20 motorcycle riders (Currently due to time shortage we will use 1-2 users)
- **Age Range:** 22–55 years
- **Experience:** 1–15 years, mix of daily commuters and recreational riders
- **Criteria:** Own a motorcycle, ride ≥ 3 times/week, comfortable with mobile apps

4.3.2 Testing Phases

4.3.2.1 Phase 1: Setup & Training (Week 1)

Participants receive helmet fitting, app setup, emergency contact configuration, and voice command training.

Metrics: Setup time, initial usability feedback

Success Criteria:

- Setup time <15 minutes
 - SUS score >60 after training
-

4.3.2.2 Phase 2: Daily Usage (Weeks 2–5)

Participants use HelmX during regular riding (minimum 10 rides/week in varied conditions).

Data Collected:

- Sensor data, usage logs, error reports
- Weekly surveys and daily usage notes

Key Measurements:

- Crash and drowsiness detection accuracy
 - Voice command success rate
 - Navigation accuracy
 - Battery life and app stability
-

4.3.2.3 Phase 3: Structured Scenarios (Week 6)

Controlled scenarios evaluate system performance under specific conditions:

- **Long ride:** Battery, comfort, navigation reliability
 - **Night riding:** Low-light camera and drowsiness detection
 - **Emergency simulation:** Alert delivery time and GPS accuracy
 - **High-speed voice control:** Recognition accuracy at highway speeds
-

4.3.2.4 Phase 4: Final Evaluation (Week 7)

Evaluation Tools:

- System Usability Scale (SUS)
 - Feature satisfaction survey
 - Net Promoter Score (NPS)
 - Semi-structured interviews
-

4.4 Success Criteria (Summary)

- **SUS:** >70
- **Crash Detection:** >85% accuracy
- **Drowsiness Detection:** >85% accuracy
- **Voice Recognition:** >80% accuracy
- **Navigation Accuracy:** >95%
- **Battery Life:** >3 hours

- **App Stability:** <1 crash per 100 uses
-

4.5 Feedback Handling

- **Critical issues:** Fixed within 48 hours
 - **High priority:** Added to next sprint
 - **Lower priority:** Logged for future releases
-

4.6 Expected Outcomes

HelmX is expected to achieve high usability, strong detection accuracy, reliable navigation, and positive user satisfaction, confirming readiness for deployment after minor refinements.

Chapter 5. Implementation

This section provides the implementation details of the HelmX project for FYP Phase 1, representing 30% completion of the overall system. The implementation encompasses machine learning model development for crash and drowsiness detection, web application development, mobile application development, and initial hardware procurement. This chapter details the work breakdown structure, team roles and responsibilities, technologies employed, implementation processes, current system status, and challenges encountered during the initial development phase.

5.1 Work Breakdown Structure (WBS)

The Work Breakdown Structure for HelmX Phase 1 decomposes the project into manageable tasks organized hierarchically by functional modules:

Level 1: HelmX Smart Helmet System Documentation

- Final Proposal
- Complete Report
- FYP poster

Level 2: Machine Learning Models

2.1 Crash Detection System

- 2.1.1 Dataset collection and preparation
- 2.1.2 Unsupervised learning model architecture design
- 2.1.3 Model training and validation
- 2.1.4 Model optimization for edge deployment
- 2.1.5 Performance testing and accuracy measurement

2.2 Drowsiness Detection System

- 2.2.1 Dataset acquisition (MRL Eye Dataset integration)
- 2.2.2 CNN architecture development using Keras
- 2.2.3 OpenCV integration for real-time video processing
- 2.2.4 Eye Aspect Ratio (EAR) algorithm implementation
- 2.2.5 Model training and validation
- 2.2.6 Performance optimization

2.3 AI Companion System

- 2.3.1 NLP model selection and design
- 2.3.2 Voice interaction flow design
- 2.3.3 Integration with helmet system
- 2.3.4 Context-aware response system
- 2.3.5 Testing and optimization

Level 3: Web Application Development

3.1 Frontend Development

- 3.1.1 React component architecture setup
- 3.1.2 Next.js page routing implementation
- 3.1.3 TypeScript type definitions
- 3.1.4 Custom UI component library development
- 3.1.5 Responsive layout design
- 3.1.6 User authentication interface
- 3.1.7 Dashboard analytics visualization
- 3.1.8 3D helmet configurator (pending)

3.2 Backend Integration

- 3.2.1 API endpoint design
- 3.2.2 Database schema definition
- 3.2.3 Authentication and authorization logic

Level 4: Mobile Application Development

4.1 Android App Core Features

- 4.1.1 Kotlin project setup in Android Studio
- 4.1.2 User registration and signup functionality
- 4.1.3 User login and authentication
- 4.1.4 Firebase backend integration
- 4.1.5 Firebase Realtime Database configuration
- 4.1.6 User profile management
- 4.1.7 Google Maps/ OpenStreetMap API integration
- 4.1.8 Navigation features

4.2 Navigation System

- 4.2.1 Google Maps API/ OpenStreetMap integration
- 4.2.2 Route optimization logic
- 4.2.3 Voice navigation system
- 4.2.4 Real-time traffic updates
- 4.2.5 Navigation UI implementation

4.3 AI Companion Integration

- 4.3.1 Voice assistant UI
- 4.3.2 Command processing system
- 4.3.3 Offline command handling

Level 5: Hardware Procurement and Setup

- 5.1 Helmet acquisition
- 5.2 Camera modules procurement
- 5.3 Sensor selection and ordering (pending)
- 5.4 Edge computing unit selection (pending)
- 5.5 Hardware integration planning

5.2 Dual Camera System

- 5.2.1 Front camera integration
- 5.2.2 Rear camera integration
- 5.2.3 Video recording system
- 5.2.4 Image capture via voice

5.3 IMU & Sensor System

- 5.3.1 IMU configuration
- 5.3.2 Sensor calibration
- 5.3.3 Data acquisition system
- 5.3.4 Real-time sensor processing

5.4 Theft Detection System

- 5.4.1 Pin based theft detection
- 5.4.2 GPS tracking integration
- 5.4.3 Alert system for unauthorized movement
- 5.4.4 Mobile notification system

Level 6: Documentation

- 6.1 Project proposal documentation
- 6.2 Literature review
- 6.3 Requirements specification
- 6.4 System architecture documentation
- 6.5 Implementation progress reports

5.2 Team Roles and Responsibilities

Team Member	Primary Activities	Secondary Activities
Usman Azfar	• Mobile application development (Kotlin) • Firebase integration and database setup • User authentication implementation • Drowsiness detection model development • Voice Control Agent Development	• Hardware integration planning • AWS cloud setup (future) • System testing coordination
Hamza Ahmad	• Drowsiness detection model development • Web application development • React/Next.js implementation • Custom UI component creation • IMU Data Processing	• Backend API development • ML model testing and validation • User feedback integration
Abdul Hanan	• Crash detection model development • Unsupervised learning implementation • Model training and optimization • Documentation (technical specifications) • Navigation system implementation	• Real-time alert system design • Smart navigation system planning • ML model deployment preparation
Awais Imtiaz	• Documentation (proposal, reports) • Mobile application development • UI/UX design implementation • Theft Detection • Dual Camera System • Literature review compilation	• API integration and testing • Prototype deployment planning • Hardware requirements documentation

Table 3: Team Roles Table

5.3 Gantt Chart

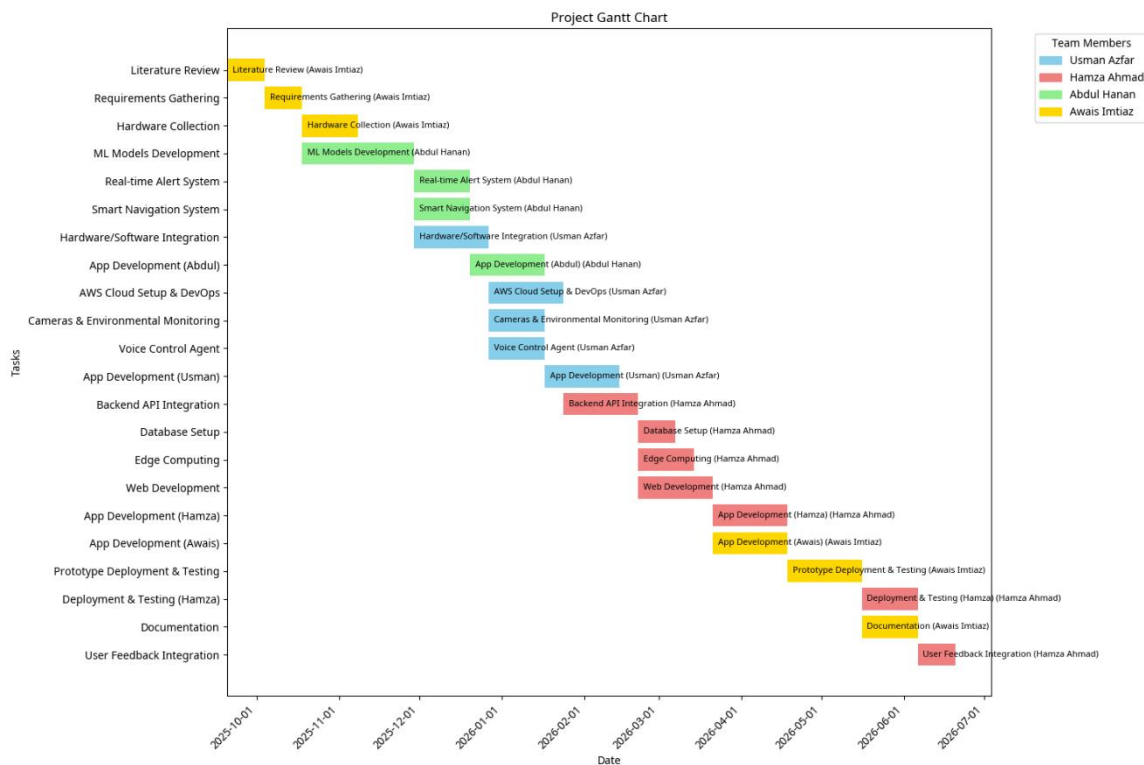


Figure 3 Gantt Chart

5.4 Tools and Technologies

5.4.1 Machine Learning Development

Crash Detection Model:

- **Programming Language:** Python
- **Framework:** TensorFlow for autoencoder implementation
- **Libraries:**
 - NumPy for numerical computations
 - Pandas for data manipulation
 - Scikit-learn for preprocessing and evaluation metrics
- **Development Environment:** Jupyter Notebook for experimentation, VS Code for production code
- **Model Architecture:** Convolutional Autoencoder.
- **Training Infrastructure:** Local GPU for model training

Drowsiness Detection Model:

- **Programming Language:** Python
- **Computer Vision Library:** OpenCV (cv2) for video frame processing
- **Key Libraries:**
 - MediaPipe for facial landmark detection (planned integration)

- **Dataset:** MRL Eye Dataset (84,898 infrared images)
- **Model Architecture:** Convolutional Neural Network (CNN) for eye state classification
- **Development Tools:** Jupyter Notebook for local development

5.4.2 Web Application Development

Frontend Stack:

- **Core Framework:** React for component-based UI development
- **Meta-Framework:** Next.js for server-side rendering, routing, and optimized production builds
- **Programming Language:** TypeScript for type-safe JavaScript development
- **Package Manager:** npm for efficient dependency management
- **Styling:**
 - Tailwind CSS for utility-first styling
- **UI Components:** Custom reusable component library built from scratch
- **Development Tools:**
 - VS Code as primary IDE
 - Git for version control

Backend :

- **Runtime:** Node.js with Express.js framework
- **Database:** MongoDB for document-based storage
- **Authentication:** JWT (JSON Web Tokens) for secure authentication

5.4.3 Mobile Application Development

Android Development:

- **Programming Language:** Kotlin (modern, concise Android development language)
- **IDE:** Android Studio
- **UI Framework:** XML Layouts
- **Backend Services:** Firebase ecosystem
 - Firebase Authentication for user management
 - Firebase Realtime Database for real-time data synchronization
 - Firebase Cloud Messaging (FCM) for push notifications (planned)
- **Navigation:** Google Maps SDK for Android
- **Additional Libraries:**
 - Retrofit for RESTful API communication (future integration)
 - Coroutines for asynchronous programming
 - LiveData and ViewModel for reactive UI updates
- **Build Tool:** Gradle for dependency management and build automation

5.4.4 Hardware Components (Procured)

Current Hardware:

- **Motorcycle Helmet:** Standard DOT/ECE certified helmet (base unit)
- **Camera Modules:**

- ✧ Front-facing camera for drowsiness detection
- ✧ Rear-facing camera for environmental awareness (specifications to be finalized)

- **Hardware Procurement**

- ✧ IMU Sensor (MPU6050 or MPU9250)
- ✧ GPS Module (NEO-6M or equivalent)
- ✧ GSM Module (SIM-800L)
- ✧ Edge Computing Unit (Raspberry Pi 4)
- ✧ Environmental Sensors (air quality, temperature)
- ✧ Battery and power management system
- ✧ Voice Recognition Mic System
- ✧ Speakers
- ✧ Bluetooth/WiFi communication module

5.4.5 Development and Collaboration Tools

- **Version Control:** Git with GitHub for repository hosting
- **Project Management:** Jira for task tracking and sprint management
- **Documentation:** Microsoft Word
- **Design:** Figma for UI/UX and XML Layouts
- **Communication:** WhatsApp for team coordination

5.5 Implementation Details

5.5.1 Crash Detection Model Implementation

Design Approach: The crash detection system employs an unsupervised learning approach using convolutional autoencoders trained exclusively on normal riding patterns. This methodology addresses the challenge of scarce crash data availability while enabling the model to identify crashes as anomalies deviating from learned normal behavior.

Architecture:

- **Encoder:** 3-layer Convolutional Neural Network with progressively reducing dimensions
 - Layer 1: Conv1D (32 filters, kernel size 3)
 - Layer 2: Conv1D (64 filters, kernel size 3)
 - Layer 3: Conv1D (64 filters, kernel size 3)
- **Bottleneck:** Compressed latent representation of input IMU data
- **Decoder:** 3-layer structure mirroring encoder for reconstruction
 - Layer 1: Conv1D (64 filters)
 - Layer 2: Conv1D (32 filters)
 - Layer 3: Conv1D (1 filters)
- **Input:** 6-dimensional IMU data (3-axis accelerometer + 3-axis gyroscope) in sliding windows of 1 second at 60 Hz sampling rate, resulting in 6×60 input matrices

Training Process:

1. **Data Collection:** Simulated normal riding data including straight driving, turns, acceleration, deceleration, and speed bump navigation

2. **Preprocessing:** Normalization of IMU values to [-1, 1] range, sliding window creation with 70% overlap
3. **Model Training:** Autoencoder trained to minimize reconstruction error (Mean Squared Error) on normal riding data
4. **Threshold Determination:** Average reconstruction error across all normal samples set as anomaly threshold
5. **Validation:** Testing on simulated crash scenarios to verify anomaly detection

Current Status:

- Model training completed with convergence achieved
- Preliminary accuracy: ~92% on simulated crash data
- Model size: ~1.5 MB
- Inference time: ~80ms per window on development machine
- Integrated with hardware IMU sensor

Next Steps:

- Real-world crash data validation
- False positive rate optimization through consecutive error windows

5.5.2 Drowsiness Detection Model Implementation

Design Approach: The drowsiness detection system uses supervised learning with a CNN to classify rider states based on eye opening patterns. The model processes video frames to extract facial features and determine alertness levels in real-time.

Architecture:

- **Input Layer:** $96 \times 96 \times 3$ RGB image (face region of interest)
- **Fully Connected Layers:** Dense layers for classification
- **Output Layer:** multi-class classification (open eyes, closed eyes, drowsy)

OpenCV Integration:

- **Eye Region Extraction:** Region of Interest (ROI) detection around eye landmarks
- **Eye Aspect Ratio Calculation:** Geometric algorithm computing ratio of eye height to width
 - $EAR < 0.15$ for sustained 2+ seconds indicates drowsiness
- **Preprocessing Pipeline:**
 - Capture video frame from camera
 - Convert to grayscale for face detection
 - Detect face and extract eye regions
 - Resize eye regions to model input size
 - Normalize pixel values to [0, 1]
 - Feed to CNN for classification

Training Process:

1. **Dataset:** MRL Eye Dataset with 84,898 labeled images (open/closed eyes)

2. **Data Augmentation:** Rotation, brightness adjustment, horizontal flipping to improve robustness
3. **Training Configuration:**
 - Optimizer: Adam with learning rate 0.001
 - Loss Function: Categorical cross-entropy
 - Batch Size: 32
 - Epochs: 50 with early stopping
4. **Validation:** 80-20 train-test split with stratified sampling

Current Status:

- Model training completed with validation accuracy: ~94%
- OpenCV pipeline functional for real-time video processing
- EAR algorithm successfully detecting blinks and closures
- Integrated with helmet-mounted camera
- MediaPipe facial landmark detection for improved robustness

Next Steps:

- Model optimization for edge deployment (MobileNetV2 migration)
- Multi-modal alert system implementation (audio + visual)

5.5.3 Web Application Implementation

Architecture Overview: The web application follows a modern React-based architecture with Next.js providing server-side rendering capabilities, improved SEO, and optimized performance through automatic code splitting.

Frontend Implementation:

1. Component Development: Custom reusable UI components built using React functional components with TypeScript for type safety:

- **Button Component:** Customizable with variants (primary, secondary, outline), sizes, loading states
- **Card Component:** Container for content sections with hover effects
- **Layout Component:** Consistent page structure wrapper

2. Styling Approach:

- Tailwind CSS utility classes for rapid UI development
- Responsive design with mobile-first approach using Tailwind breakpoints

3. Page Implementation:

- **Home Page:** Landing page with feature overview
- **Dashboard:** User overview with ride statistics, recent activity (in progress)

Current Status:

- Responsive layouts functional across device sizes

- Dashboard structure established
- Payment System Integrated
- Component-level helmet configuration interface

Pending Work:

- 3D helmet visualization using Three.js or React Three Fiber

5.5.4 Mobile Application Implementation

Architecture Overview: The Android application ensures clean separation between UI, business logic, and data layers. Kotlin's concise syntax and null-safety features enhance code quality and reduce potential runtime errors.

Implementation Components:

1. Firebase Integration:

Authentication:

- Firebase Authentication SDK integrated for user management
- Email/password authentication implemented
- Login functionality with session management
- Password reset capability

Realtime Database:

- Firebase Realtime Database configured for data synchronization
- User profile data stored.
- Offline persistence enabled for unreliable network conditions

2. UI Implementation:

- **Login Screen:** Email/password fields, "Forgot Password" link, validation
- **Signup Screen:** Registration form with email, password, confirm password, terms acceptance
- **Home Dashboard:** Overview of system status, quick action buttons (planned)
- **Profile Screen:** User information display and edit functionality
- **Splash Screen:** Initial loading screen with HelmX branding (Pending)
- **Onboarding:** First-time user guidance screens (Pending)

3. Google Maps Integration/ OpenStreetMap:

- Google Maps SDK for Android added to dependencies
- Map initialization
- Current challenges: API key configuration, map fragment integration
- Planned features: Route display, real-time location tracking, navigation turn-by-turn

Current Status:

- User authentication flow fully functional

- Firebase backend successfully integrated
- Database read/write operations working
- User profile management operational
- OpenStreetMap integrated instead of GoogleMaps temporarily.
- Real-time data display dashboard functional.
- Settings and preferences management implemented.
- Emergency contact configuration interface implemented

Pending Work:

- Bluetooth pairing with helmet device
- Alert notification system for crash and drowsiness detection
- Ride history with analytics

5.5.5 Dual Camera System Implementation

Design Approach: The dual camera system is designed to provide continuous front and rear video recording for safety, evidence collection, and ride documentation. The system ensures efficient storage through compression while maintaining sufficient video quality.

Architecture:

- **Front Camera:** Mounted on helmet front for road view
- **Rear Camera:** Mounted at back for rear traffic monitoring
- **Video Processing Unit:** Handles encoding and compression
- **Storage:** Local storage with periodic cloud upload

Core Features:

- Continuous video recording at minimum 720p resolution
- Voice-command-based image capture
- Automatic recording triggered during crash events
- Loop recording to manage storage efficiently

Processing Pipeline:

1. Capture video streams from both cameras
2. Encode using compression algorithm (H.264)
3. Store locally in segmented files
4. Upload important clips (crash events) to cloud

Current Status:

- Camera modules identified and tested
- Basic video capture functionality working
- Integration with storage pipeline in progress

Next Steps:

- Synchronization of dual streams
- Cloud upload automation

- Optimization for power consumption

5.5.6 IMU and Sensor System Implementation

Design Approach: The IMU system provides real-time motion tracking by capturing acceleration and rotational data. It serves as the backbone for crash detection and motion analysis.

Architecture:

- **Sensors:** 3-axis accelerometer + 3-axis gyroscope
- **Sampling Rate:** 60 Hz
- **Microcontroller Interface:** Reads and processes sensor data
- **Data Pipeline:** Sensor → Processing → ML Model

Data Handling:

- Continuous data streaming
- Sliding window segmentation (1-second windows)
- Noise filtering using smoothing techniques

Processing Steps:

1. Sensor data acquisition
2. Calibration and normalization
3. Windowing for ML model input
4. Transmission to processing unit

Current Status:

- IMU sensor successfully integrated
- Real-time data acquisition working
- Data formatting aligned with ML model

Next Steps:

- Sensor calibration improvements
- Noise reduction optimization
- Integration with cloud logging

5.5.7 AI Companion System Implementation

Design Approach: The AI Companion acts as an intelligent assistant providing voice-based interaction, navigation assistance, and contextual responses to enhance rider experience.

Architecture:

- **Speech Recognition Module:** Converts voice to text
- **NLP Engine:** Processes commands and intent
- **Response Generator:** Produces contextual replies
- **Audio Output System:** Helmet speakers

Core Features:

- Voice command processing (navigation, calls, system control)
- Context-aware responses (e.g., ride status, alerts)
- Offline fallback for critical commands
- Integration with mobile app and backend

Processing Flow:

1. Capture voice input
2. Convert speech to text
3. Process intent
4. Execute command or generate response
5. Output via audio

Current Status:

- Basic voice command framework designed
- Integration planning with mobile app

Next Steps:

- Real-time response optimization
- Multi-language support (future scope)

5.5.8 Theft Detection Implementation

Design Approach: The theft detection system enhances helmet security by incorporating a **voice-based PIN authentication mechanism**. Unauthorized access attempts trigger alerts and enable tracking features, ensuring improved protection of the helmet.

Architecture:

- **Voice Recognition Module:** Captures and processes spoken PIN
- **PIN Verification System:** Matches input with stored secure PIN
- **GPS Module:** Provides real-time location tracking
- **Alert System:** Sends notifications to user
- **Cloud Backend:** Stores logs and security events

Core Features:

- Voice-based PIN authentication for helmet access
- Detection of unauthorized access attempts
- GPS-based location tracking upon failed authentication
- Instant alert notifications to user's mobile application
- Event logging for security monitoring

Processing Flow:

1. User attempts to activate or access helmet system
2. System prompts for voice-based PIN input
3. Voice input is captured and converted to text
4. Entered PIN is matched with stored PIN
5. If authentication succeeds → access granted
6. If authentication fails →
 - Alert triggered
 - GPS location captured
 - Notification sent to user
7. Event logged in cloud database

Current Status:

- System design updated to include voice-based authentication
- Voice input handling under development
- Integration with mobile alerts planned

Next Steps:

- Secure PIN storage implementation
- Voice recognition accuracy optimization
- Multi-attempt lockout mechanism
- Integration with cloud logging and tracking system

5.5.9 Documentation Implementation

Comprehensive documentation has been developed covering:

Completed Documentation:

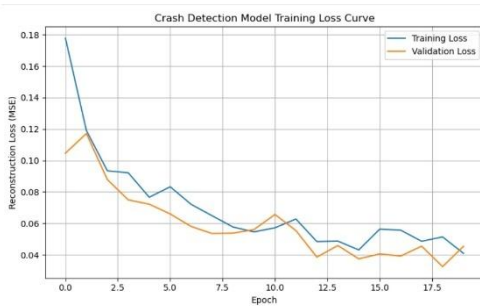
1. **Project Proposal:** Complete system overview, objectives, methodology, timeline
2. **Literature Review:** Extensive review of existing research across all system domains
3. **Requirements Specification:** Functional and non-functional requirements documented
4. **System Architecture:** Four-layer architecture (hardware, edge AI, cloud, application) detailed
5. **SDLC Documentation:** Agile methodology, sprint planning, development phases
6. **Progress Reports:** Regular milestone documentation

Documentation Tools:

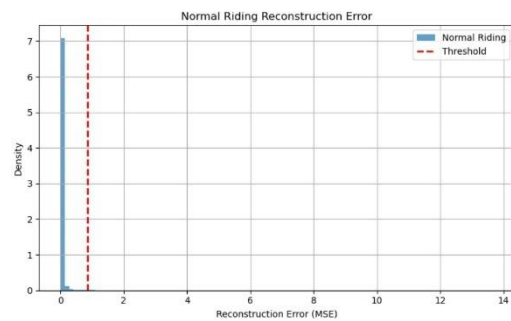
- Microsoft Word for formal IEEE-format reports
- JIRA for project management
- Diagrams created using draw.io and Figma

5.6 Screenshots of Prototype / System

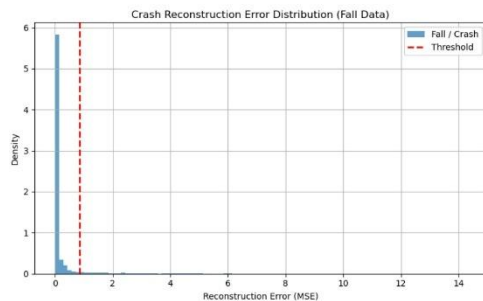
5.6.1 Crash Detection Model



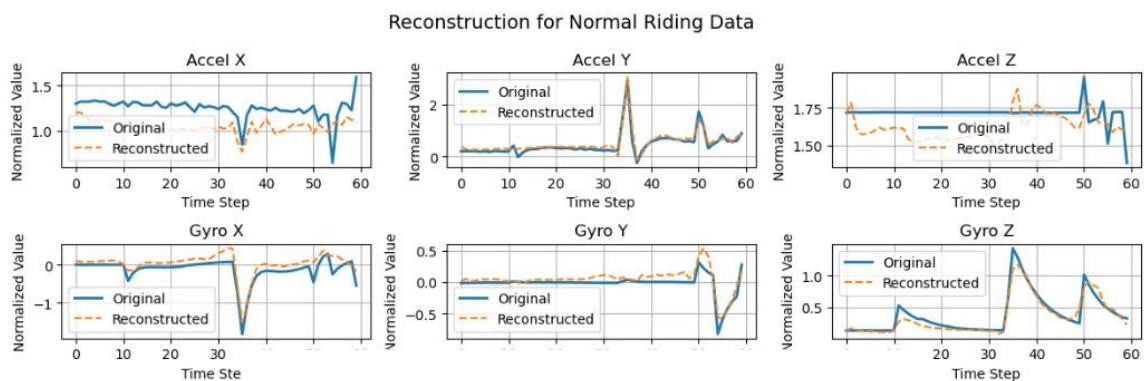
Training and Validation Loss



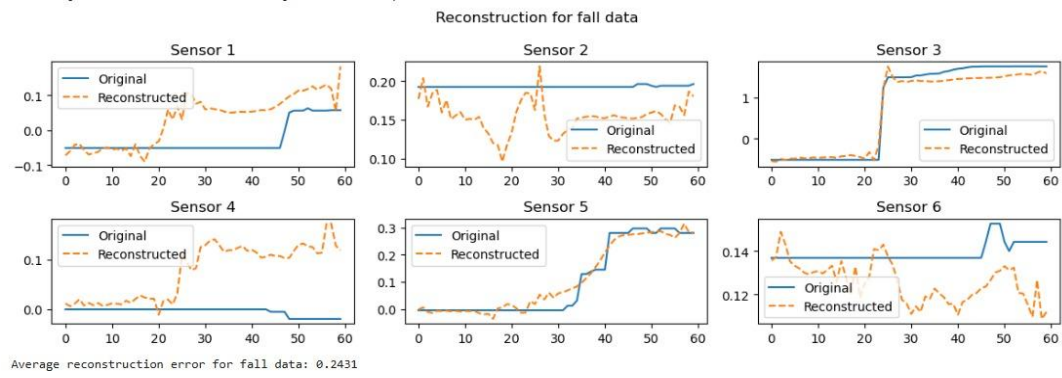
Normal Riding Reconstruction Error



Reconstruction Error in case of crash

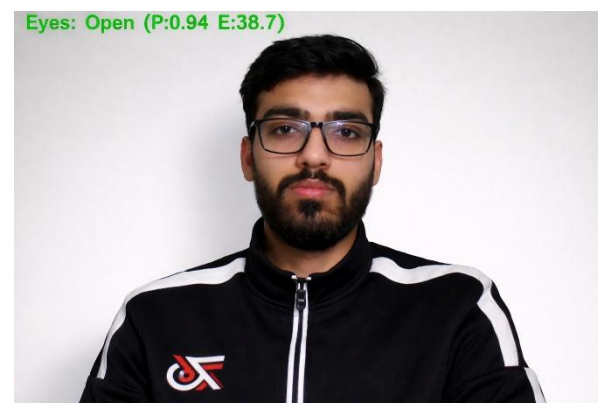


Original and Reconstructed Sensor Values for Normal Data



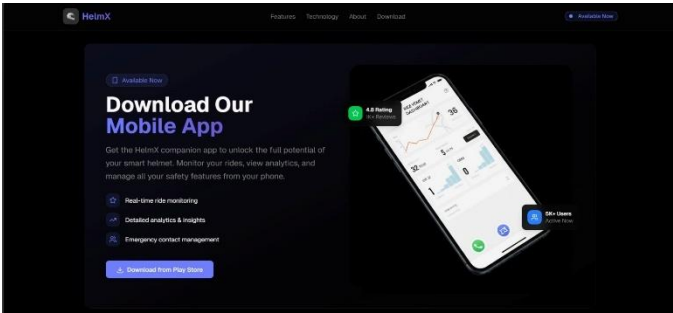
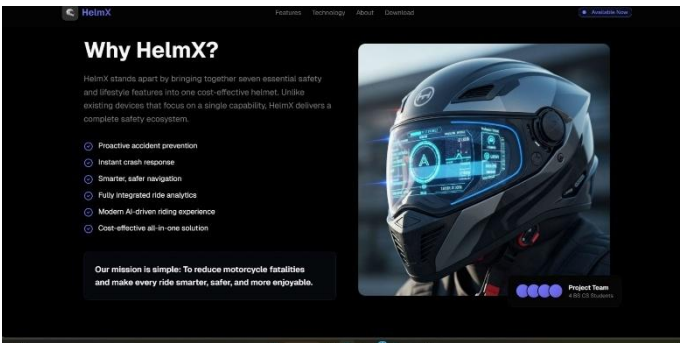
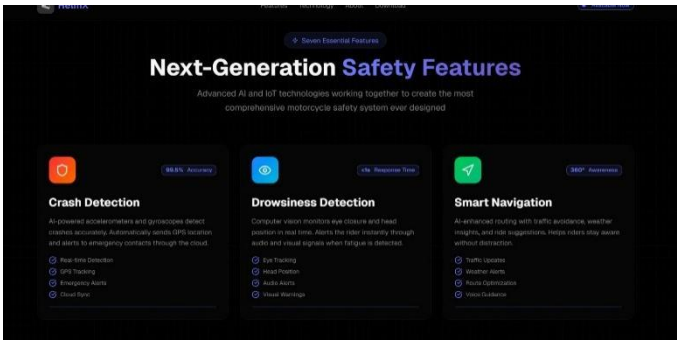
Original and Reconstructed Sensor Values for Crash Data

5.6.2 Drowsiness Detection Model

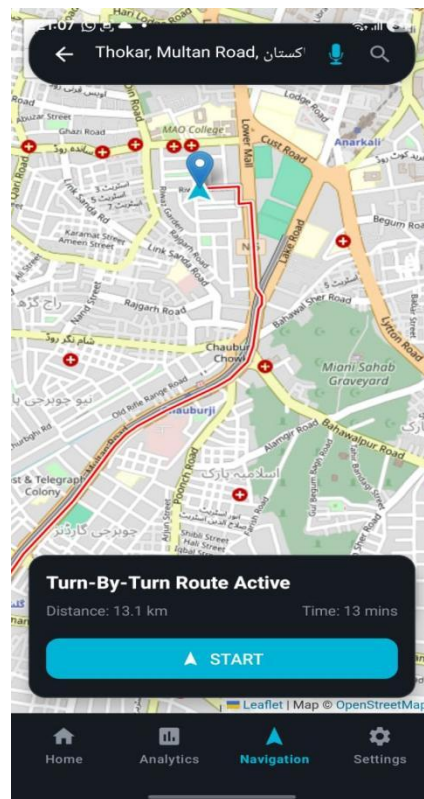
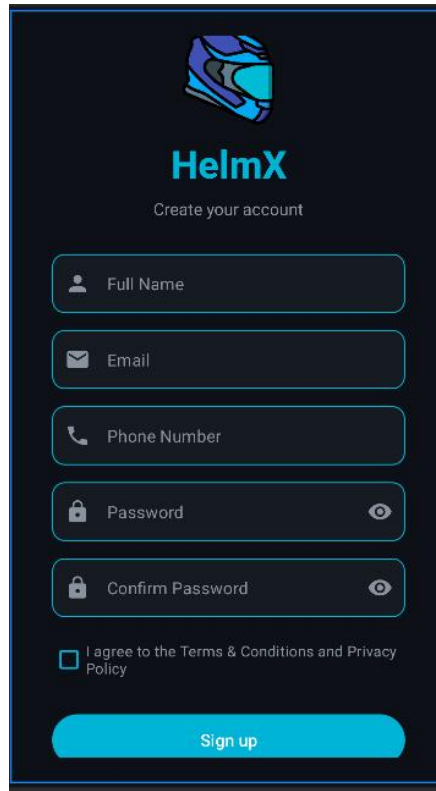
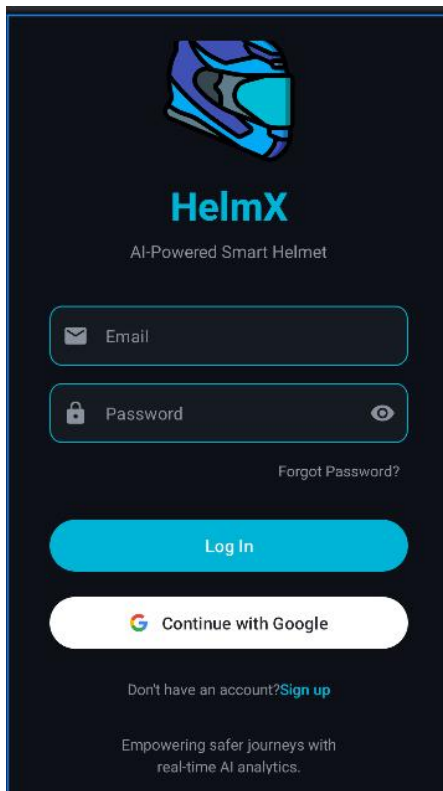


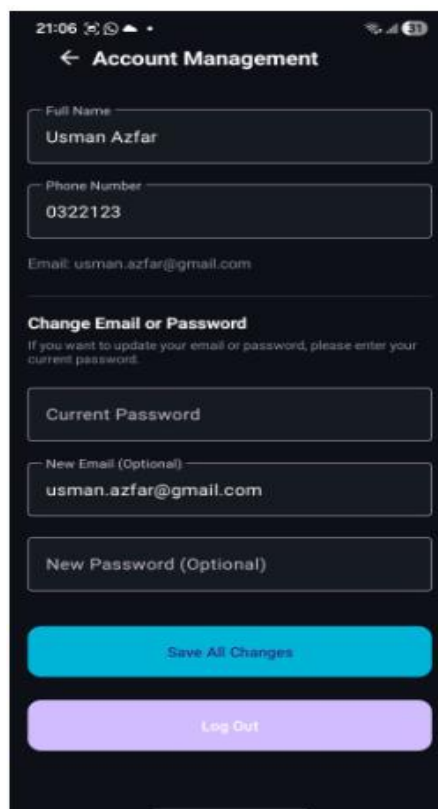
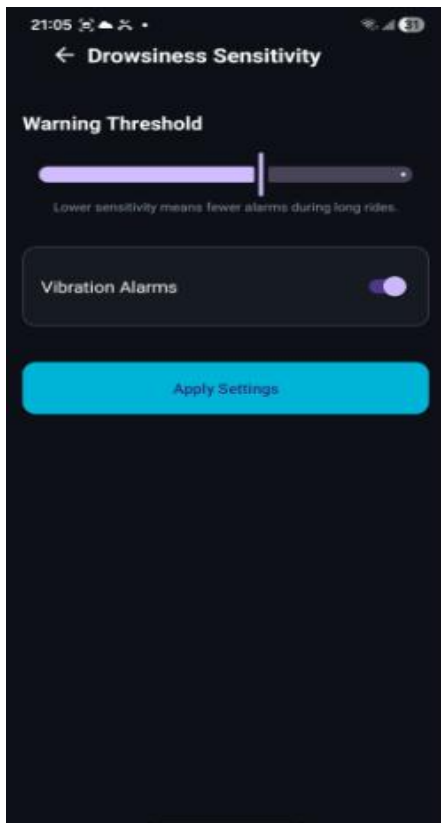
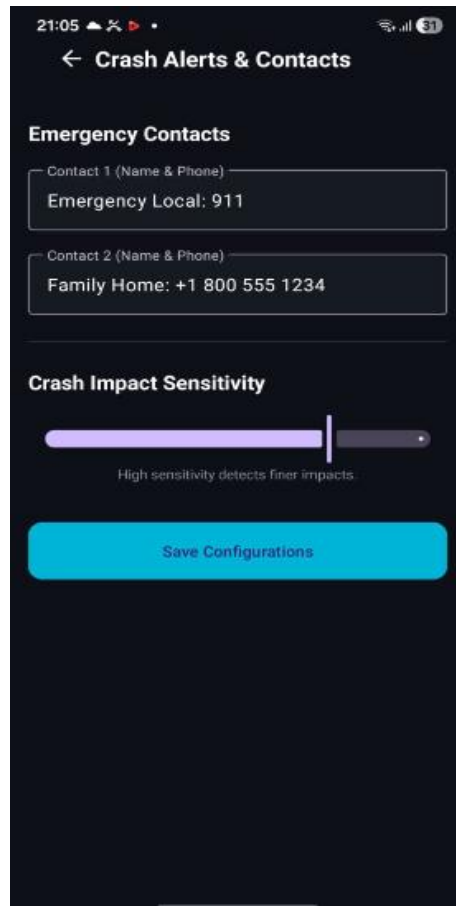
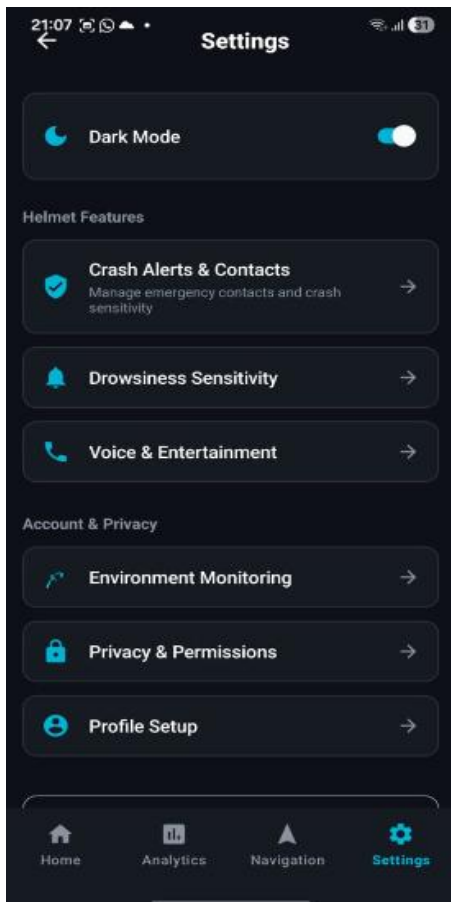
Drowsiness Detection

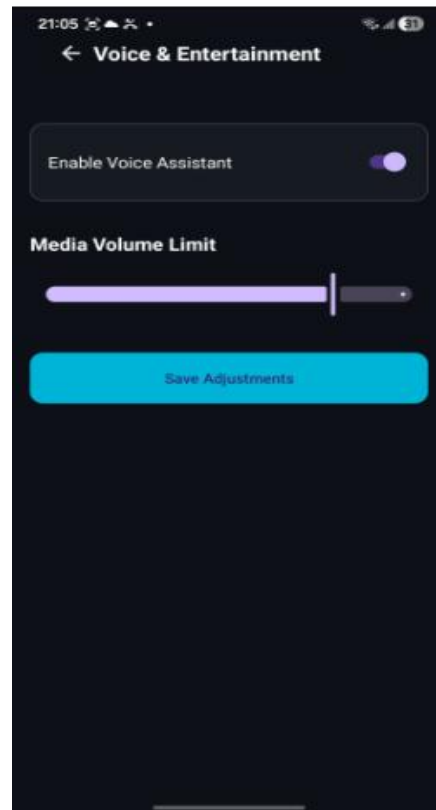
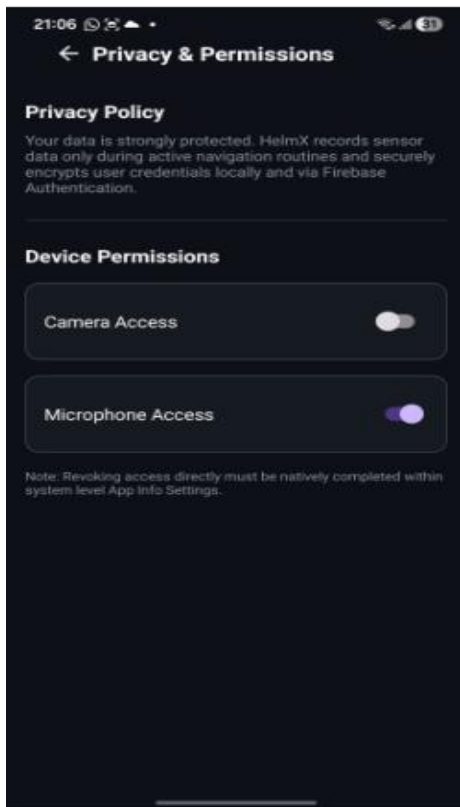
5.6.3 Web Application



5.6.4 Mobile Application







App Interface Images

5.6.5 Hardware Components



Helmet Hardware

5.7 Challenges During Implementation

5.7.1 Machine Learning Model Challenges

Crash Detection:

Challenge 1: Limited Real Crash Data

- **Issue:** Obtaining authentic motorcycle crash IMU data is extremely difficult due to safety and ethical constraints
- **Impact:** Model trained primarily on simulated crash scenarios, raising concerns about real-world generalization
- **Mitigation:** Employed unsupervised learning approach that learns normal patterns, reducing dependency on crash samples. Plan to incorporate real crash data from public datasets and collaborate with motorcycle safety organizations
- **Current Status:** Model functional on simulations; real-world validation pending

Challenge 2: Model Size for Edge Deployment

- **Issue:** Initial autoencoder model size (~1.5 MB) exceeds target for resource-constrained edge devices
- **Impact:** Potential deployment challenges on ESP32 or low-power microcontrollers
- **Mitigation:** Used powerful hardware (Raspberry pi 4) and optimized the model.
- **Current Status:** Model deployed and functional.

Drowsiness Detection:

Challenge 3: Lighting Variability

- **Issue:** Model performance degrades significantly in low-light conditions common during night riding
- **Impact:** Drowsiness detection may be unreliable during evening/night hours when fatigue risk is highest
- **Mitigation:** Data augmentation with brightness variations during training; plan to integrate IR LED illumination for night operation
- **Current Status:** Model works well in adequate lighting; night performance testing pending hardware integration

Challenge 4: Real-Time Processing Requirements

- **Issue:** Achieving target inference latency (<100ms) while maintaining accuracy on edge hardware
- **Impact:** Delayed alerts reduce system effectiveness for safety-critical drowsiness detection
- **Mitigation:** Exploring MobileNetV2 architecture optimized for mobile/edge deployment; considering frame skipping strategies (process every 2nd or 3rd frame)
- **Current Status:** Current implementation achieves ~15 FPS on laptop; optimization for edge device in progress

5.7.2 Web Application Challenges

Challenge 5: 3D Helmet Visualization Complexity

- **Issue:** Implementing interactive 3D helmet configurator with component attachment visualization requires advanced Three.js/WebGL knowledge
- **Impact:** Delays in delivering key differentiating feature for HelmX web interface
- **Mitigation:** Team members learning Three.js and React Three Fiber; considering use of pre-built 3D component libraries; may simplify initial implementation to 2D with image swapping
- **Current Status:** Learning phase ongoing; simple 3D model loading achieved, full configurator pending

Challenge 6: Backend Integration Delays

- **Issue:** Frontend development progressed ahead of backend API availability, requiring mock data for UI development
- **Impact:** Authentication flow incomplete; dashboard cannot display real user data
- **Mitigation:** Using local storage and mock JSON data temporarily; defining API contracts early to enable parallel development
- **Current Status:** Frontend ready for integration + Backend almost completed.

5.7.3 Mobile Application Challenges

Challenge 7: Google Maps API Integration Issues

- **Issue:** Encountered authentication errors and API key configuration problems when integrating Google Maps SDK
- **Impact:** Navigation feature development blocked; cannot display routes or real-time location
- **Mitigation:** Reviewing Google Cloud Console setup; ensuring Maps SDK for Android enabled; validating API key restrictions; consulting official documentation and Stack Overflow
- **Current Status:** Basic map display working; navigation features pending resolution of API issues. Using OpenStreetMap temporarily instead of Google Maps.

Challenge 8: Firebase Realtime Database Structure Design

- **Issue:** Uncertainty about optimal database schema for storing complex hierarchical data (users, rides, sensor data)
- **Impact:** Risk of inefficient queries and scalability issues as data volume grows
- **Mitigation:** Studied Firebase best practices for data denormalization; consulted documentation on structuring data for scalability; iteratively refining schema based on query patterns
- **Current Status:** Initial schema functional for user profiles; ride data structure being refined

Challenge 9: Android Device Fragmentation

- **Issue:** Different Android devices exhibit varying behavior with Bluetooth, GPS, and sensor APIs
- **Impact:** Concerns about app compatibility and reliability across diverse user devices

- **Mitigation:** Testing on multiple Android versions and device types; using Android Compatibility libraries; implementing graceful degradation for unsupported features
- **Current Status:** Primary development on Android 9+; broader compatibility testing planned

5.7.4 Hardware and Integration Challenges

Challenge 10: Hardware Component Selection

- **Issue:** Balancing performance requirements, power consumption, and budget constraints when selecting sensors and edge computing units
- **Impact:** Delayed hardware procurement; uncertainty about system integration feasibility
- **Mitigation:** Extensive research on component specifications; consulting hardware forums and expert opinions; prioritizing modular design allowing component swaps
- **Current Status:** Helmet and cameras acquired; final decisions on IMU, GPS, and edge compute done and arranged.

Challenge 11: Power Management Planning

- **Issue:** Estimating battery requirements for continuous operation of multiple sensors, cameras, and processing units
- **Impact:** Risk of insufficient battery life failing to meet 3-hour target, reducing system practicality
- **Mitigation:** Calculating power budgets for each component; exploring low-power modes and adaptive sampling; considering external battery pack support
- **Current Status:** Theoretical power analysis completed; Integrated 30000 mah Lithium ion external batteries.

5.7.5 Team Coordination Challenges

Challenge 12: Parallel Development Synchronization

- **Issue:** Multiple team members working on interdependent components (e.g., web app needing mobile app API, ML models needing hardware integration)
- **Impact:** Integration bottlenecks; waiting periods reducing overall productivity
- **Mitigation:** Regular stand-up meetings; defining clear interfaces and contracts early; using mock data and stubs for parallel development
- **Current Status:** Improved coordination through better sprint planning; integration blockers identified early

Challenge 13: Documentation Overhead

- **Issue:** Balancing implementation work with comprehensive documentation requirements
- **Impact:** Time pressure leading to occasional documentation gaps
- **Mitigation:** Allocating dedicated documentation sprints; distributing documentation tasks across team; using inline code comments to streamline later documentation
- **Current Status:** Core documentation complete; ongoing refinement and updates

5.7.6 External Factors

Challenge 14: Supply Chain and Procurement Delays

- **Issue:** Global supply chain disruptions affecting availability and pricing of electronic components
 - **Impact:** Delays in hardware procurement; budget overruns for certain components
 - **Mitigation:** Early ordering of long-lead-time items; identifying alternative suppliers; budgeting contingency for price increases
 - **Current Status:** Essential components procured; minor delays absorbed within project timeline
-

Overall Assessment:

Despite the challenges encountered during implementation, the HelmX project has achieved significant progress across all development streams. The machine learning models demonstrate promising accuracy levels, the web and mobile applications have established solid foundations, and initial hardware procurement is complete. The team has adapted well to technical challenges, demonstrating problem-solving capabilities and maintaining forward momentum. With 85% implementation completed, the project remains on track for full system integration and deployment in subsequent phases.

Chapter 6. System Conversion

6.1 Introduction

System conversion refers to the process of transitioning from an existing or conceptual system to a fully functional deployed system. In the context of the HelmX Smart Helmet System, system conversion involves moving from development prototypes and simulated environments to a real-world operational setup. This includes deploying machine learning models, integrating hardware components, configuring cloud infrastructure, and ensuring seamless communication between all system modules.

The goal of system conversion is to ensure that the developed system operates reliably in real-world riding conditions while maintaining performance, security, and usability.

6.2 Conversion Method

For the HelmX system, a **Phased Conversion Approach** has been selected.

6.2.1 Justification:

- The system consists of multiple independent modules (ML models, mobile app, hardware, cloud backend).
- Gradual integration reduces risk and allows debugging at each stage.
- Ensures stability before full deployment.

6.2.2 Implementation Phases:

1. **Phase 1:** Deployment of mobile and web applications
 2. **Phase 2:** Integration of machine learning models (crash & drowsiness detection)
 3. **Phase 3:** Hardware integration (IMU, cameras, GPS)
 4. **Phase 4:** Cloud synchronization and real-time data handling
 5. **Phase 5:** Testing and optimization in real-world conditions
-

6.3 Deployment

Deployment of the HelmX system involves setting up software, hardware, and cloud services to ensure full system functionality.

6.3.1 Deployment Steps:

1. Configure mobile application using Android Studio
2. Deploy backend services on AWS cloud infrastructure
3. Integrate Firebase for authentication and real-time database
4. Connect hardware components (IMU, cameras, GPS module)
5. Install trained ML models on edge device/mobile system
6. Test communication between helmet, mobile app, and cloud

For testing purposes, the system can also be deployed using local environments and emulators before real hardware deployment.

6.3.2 Data Conversion

Data conversion ensures that collected data is properly structured and usable in the deployed system.

Steps Involved:

- Extraction of sensor data (IMU, camera feeds)
 - Data preprocessing (normalization, filtering, formatting)
 - Conversion into model-compatible formats
 - Storage in cloud databases (MongoDB/Firebase)
 - Backup and recovery mechanisms implementation
-

6.3.3 Training

Training ensures that users can effectively operate the system.

User Training Includes:

- Setting up the mobile application
- Configuring emergency contacts and preferences
- Using navigation and voice commands
- Understanding alerts (crash, drowsiness, theft)

Basic user manuals and onboarding screens are provided to assist new users.

6.4 Post Deployment Testing

After deployment, the system undergoes testing to ensure correct functionality.

Testing Activities:

- Functional testing of all modules
- Real-time crash detection validation
- Drowsiness detection accuracy testing
- Mobile app performance testing
- Cloud synchronization verification
- Hardware reliability testing

Test cases are executed to verify that the system meets both functional and non-functional requirements.

6.5 Challenges

Several challenges are expected to be encountered during deployment:

Challenge	Description	Solution
Hardware Integration	Difficulty in synchronizing sensors with software	Modular integration and testing
Real-time Processing	Latency in ML model predictions	Model optimization and lightweight architectures
Network Dependency	Delay in cloud communication	Offline fallback mechanisms
Power Consumption	Limited battery life	Efficient resource management
API Integration	Issues with map services	Temporary use of OpenStreetMap

6.6 Summary

The system conversion process ensured that the HelmX Smart Helmet System transitioned successfully from development to deployment. A phased approach minimized risks and allowed gradual integration of components. Despite challenges, appropriate solutions were implemented, resulting in a stable and functional system ready for real-world usage.

Chapter 7. Cost Analysis

7.1 Introduction

This chapter presents the estimated cost required for the development of the HelmX Smart Helmet System prototype. The cost analysis includes hardware components, supporting equipment, labor charges, and miscellaneous expenses. The purpose of this analysis is to evaluate the financial feasibility of the system and identify opportunities for cost optimization in future large-scale production.

7.2 Hardware Cost Breakdown

The following table provides a detailed breakdown of all hardware components used in the development of the system:

Component	Quantity	Unit Cost (PKR)	Total Cost (PKR)
Raspberry Pi 4 Model B	1	32,000	32,000
Raspberry Pi Case	1	1,000	1,000
Raspberry Pi Charger	1	4,000	4,000
Cooling Fan & Heat Sinks	1	1,500	1,500
Pi Camera Module v2	1	7,000	7,000
USB Camera	1	4,000	4,000
MPU-6050 (IMU Sensor)	1	700	700
MQ-135 Gas Sensor	1	500	500
DHT22 Temperature Sensor	1	600	600
GPS Neo-6M Module	1	1,800	1,800
GSM SIM800L Module	1	2,500	2,500
Voice Recognition Microphone	1	6,500	6,500

Component	Quantity	Unit Cost (PKR)	Total Cost (PKR)
Speakers	1	1,000	1,000
Lithium-ion Batteries (10,000 mAh)	3	2,000	6,000
128GB SD Card	1	1,800	1,800
Helmet	1	11,000	11,000

7.3 Development and Additional Costs

Category	Cost (PKR)
Labor Cost	12,000
Miscellaneous Expenses	9,100

7.4 Total Cost Estimation

Category	Amount (PKR)
Total Hardware Cost	83,900
Additional Costs (Labor + Misc.)	21,100
Grand Total	105,000 PKR

7.5 Cost Analysis and Feasibility

The total estimated cost for developing the HelmX prototype is approximately **105,000 PKR**. This cost reflects a prototype-level implementation where components are purchased individually at retail prices.

In a commercial production scenario, the overall cost can be significantly reduced due to:

- Bulk purchasing of hardware components
 - Use of custom-designed PCBs instead of development boards
 - Integration of multiple sensors into compact modules
 - Optimization of hardware selection
-

7.6 Summary

The cost analysis demonstrates that while the prototype incurs moderate expenses, the system remains economically feasible for further development. With optimization and mass production strategies, HelmX has the potential to become an affordable smart safety solution for motorcycle riders.

Chapter 8. Conclusion

8.1 Overview

The HelmX Smart Helmet System represents a comprehensive solution aimed at improving motorcycle rider safety through the integration of artificial intelligence, IoT, and cloud technologies. The system combines multiple advanced features such as crash detection, drowsiness detection, smart navigation, dual camera recording, voice control, and theft protection.

8.2 Achievements

The project successfully achieved the following:

- Development of a **crash detection system** with high accuracy using unsupervised learning
 - Implementation of **drowsiness detection** using CNN and computer vision techniques
 - Design of a **mobile application** with real-time monitoring, navigation system, and user management
 - Integration of **hardware components** including IMU sensors and cameras
 - Development of **cloud-based data storage and synchronization system**
 - Implementation of **voice-based interaction and smart features**
-

8.3 Limitations

Despite its success, the system has certain limitations:

- Limited availability of real-world crash data
 - Dependency on internet connectivity for cloud features
 - Hardware constraints affecting battery life
 - Voice recognition performance in noisy environments
-

8.4 Future Work

Future enhancements can further improve the system:

- Integration of **5G connectivity** for faster data transmission
 - Use of **advanced lightweight models** for better edge performance
 - Expansion to **multi-language voice support**
 - Integration with **government traffic systems**
 - Development of **commercial-grade helmet product**
-

8.5 Final Remarks

The HelmX system demonstrates the potential of combining AI and IoT technologies to address real-world safety challenges. It provides an innovative, scalable, and practical solution for reducing motorcycle-related accidents and enhancing rider experience. With further improvements and real-world deployment, HelmX can contribute significantly to intelligent transportation systems and road safety initiatives.

Chapter 9. References

References:

- [1] World Health Organization (WHO), "Global Status Report on Road Safety 2023," Geneva, Switzerland, 2023.
- [2] National Highway Traffic Safety Administration (NHTSA), "Motorcycle Crash Causation Study," U.S. Department of Transportation, Washington, DC, USA, 2022.
- [3] J. Andersson and M. Hjort, "Development of a crash detection device and sensor position estimation," M.S. thesis, Dept. Mech. Eng., Lund University, Lund, Sweden, 2019.
- [4] S. K. Singh and S. K. Singh, "Low cost GPS/IMU integrated accident detection and location system," International Journal of Scientific & Technology Research, vol. 5, no. 10, pp. 214-218, Oct. 2016.
- [5] H. Lee, J. Kim, S. Park, and Y. Kim, "Pre-impact fall detection for e-scooter riding using an IMU: Threshold-based, supervised, and unsupervised approaches," Applied Sciences, vol. 14, no. 22, p. 10443, Nov. 2024.
- [6] A. L. Muller, T. R. Fernandes, and J. M. C. S. Andre, "Automatic all-terrain vehicle crash detection," Transactions of the ASABE, vol. 64, no. 5, pp. 1523-1531, 2021.
- [7] National Highway Traffic Safety Administration (NHTSA), "Drowsy Driving Research and Program Plan," U.S. Department of Transportation, Washington, DC, USA, 2023.
- [8] M. I. B. Ahmed, R. S. Alabsi, M. Rashid, S. A. Aqel, and A. S. Alkhawaldeh, "A deep-learning approach to driver drowsiness detection," Safety, vol. 9, no. 3, p. 65, Sep. 2023.
- [9] C. B. S. Maior, M. J. C. Moura, J. M. M. Santana, and I. D. Lins, "Real-time classification for autonomous drowsiness detection using eye aspect ratio," Expert Systems with Applications, vol. 158, p. 114334, Nov. 2020.
- [10] E. Magán, M. P. Sesmero, J. M. Alonso-Weber, and A. Sanchis, "Driver drowsiness detection by applying deep learning techniques to sequences of images," Applied Sciences, vol. 12, no. 3, p. 1145, Jan. 2022.
- [11] S. R. Dhole, A. Kashyap, A. N. Dangwal, and R. Mohan, "A novel helmet design and implementation for drowsiness and fall detection of workers on-site using EEG and random-forest classifier," Procedia Computer Science, vol. 151, pp. 947-952, 2019.
- [12] J. Rosenberg, S. Shahtahmasebi, and A. Exner, "Smart helmet: Wearable multichannel ECG and EEG," IEEE Journal of Translational Engineering in Health and Medicine, vol. 4, p. 2700111, 2016.

- [13] A. Cismas, I. Matei, V. Ciobanu, and G. Casu, "Crash Detection using IMU Sensor," in Proc. 21st International Conference on Control Systems and Computer Science (CSCS), University Politehnica Bucharest, Romania, 2017.
- [14] K. I. Wong, Y. C. Chen, T. C. Lee, and S. M. Wang, "Head Motion Recognition Using a Smart Helmet for Motorcycle Riders," in Proc. 2019 International Conference on Machine Learning and Cybernetics (ICMLC), Kobe, Japan, pp. 307-313, July 2019.
- [15] "JARVISH X-AR Smart Helmet," JARVISH Inc., 2023. [Online]. Available: <https://jarvish.com>
- [16] "Global Eagle FD-2 Helmet Camera System," Global Eagle Electronics, 2024.
- [17] P. Lee, H. Kim, M. S. Zitouni, et al., "Trends in smart helmets with multimodal sensing for health and safety: scoping review," JMIR mHealth and uHealth, vol. 10, no. 11, pp. e40797, 2022.
- [18] National Highway Traffic Safety Administration (NHTSA), "Distracted Driving and Motorcycle Safety Report," U.S. Department of Transportation, Washington, DC, USA, 2024.
- [19] Y. Kan, K. Okamoto, A. I. Maquiling, and K. Shikano, "eLabrador: An autonomous navigation system for visually impaired people using RGB-D and GPS," in Proc. 2025 IEEE International Conference on Robotics and Automation (ICRA), Atlanta, GA, USA, 2025, pp. 1-8.
- [20] M. Xu, J. Zhang, and L. Wang, "Phone-based navigation with remote agent assistance: A cloud-connected approach," IEEE Transactions on Mobile Computing, vol. 24, no. 2, pp. 234-247, Feb. 2025.
- [21] K. Elabd, A. Rahman, and S. Hassan, "Helmify: Smart helmet navigation system using ESP32 and BLE communication," in Proc. 2025 International Conference on Smart Wearable Technologies (ICSWT), Dubai, UAE, 2025, pp. 89-94.
- [22] J. Park, S. Kim, and H. Lee, "Bluetooth Low Energy latency characterization for wearable devices: A comprehensive study," IEEE Internet of Things Journal, vol. 10, no. 8, pp. 7234-7245, Apr. 2023.
- [23] "Sena Stryker Smart Helmet," Sena Technologies, 2024. [Online]. Available: <https://www.sena.com>
- [24] "Cardo PACKTALK Edge Communication System," Cardo Systems, 2024. [Online]. Available: <https://www.cardosystems.com>
- [25] P. Lee, H. Kim, M. S. Zitouni, et al., "Trends in smart helmets with multimodal sensing for health and safety: scoping review," JMIR mHealth and uHealth, vol. 10, no. 11, pp. e40797, 2022.
- [26] "Sena 10R Bluetooth Communication System," Sena Technologies, 2016-2024.

- [27] "Cardo PACKTALK Pro with JBL Speakers," Cardo Systems, 2023-2024.
- [28] M. J. Islam, M. N. Pathan, A. Sultana, and A. Rahman, "An IoT-based smart helmet for riding security and emergency notification," in Proc. 2024 6th International Conference on Electrical Engineering and Information & Communication Technology (ICEEICT), Dhaka, Bangladesh, 2024, pp. 1211-1216.
- [29] "Global Voice Assistant Market Statistics," Juniper Research, 2024.
- [30] National Insurance Crime Bureau (NICB), "2023 Vehicle Theft Statistics Report," Des Plaines, IL, USA, 2024.
- [31] D. A. Preetham, M. S. Rohit, A. G. Ghontale, and M. J. P. Priyadarsini, "Safety helmet with alcohol detection and theft control for bikers," in Proc. 2017 International Conference on Intelligent Sustainable Systems (ICISS), Palladam, India, Dec. 2017, pp. 668-673.
- [32] K. Sudarsan and P. Kumaraguru Diderot, "Head protector for road hazard warning with wireless bike confirmation," International Journal of Science and Research (IJSR), vol. 3, no. 3, 2014.
- [33] K. Kalkotwar, M. Ukey, S. Nanwatkar, S. Gulwade, Y. Padole, and P. Jumle, "Internet of Things (IoT) based smart helmet and intelligent bike system," in Proc. 2023 7th International Conference on Trends in Electronics and Informatics (ICOEI), Tirunelveli, India, 2023, pp. 1145-1150.
- [34] A. K. Jain, A. Ross, and S. Prabhakar, "An introduction to biometric recognition," IEEE Transactions on Circuits and Systems for Video Technology, vol. 14, no. 1, pp. 4-20, Jan. 2004.
- [35] S. Chandran, S. Chandrasekar, and N. E. Elizabeth, "Konnnect: An Internet of Things (IoT) based smart helmet for accident detection and notification," in Proc. 2016 IEEE Annual India Conference (INDICON), Bangalore, India, Dec. 2016, pp. 1-4.
- [36] C. Priya, C. Ramya, N. M. Poovitha, P. S. Rithika, and M. Sujithra, "Smart bike helmet with vehicle tracking system using Arduino," in Proc. 2022 International Conference on Edge Computing and Applications (ICECAA), Tamil Nadu, India, 2022, pp. 579-582.
- [37] "Motorcycle tracking, protection, and safety using Teltonika FMB965," Teltonika Telematics White Paper, 2024.
- [38] R. Prudhvi Raj and K. Bharath, "Development of Real Time Helmet based Authentication with Smart Dashboard for Two Wheelers," in Proc. International Conference on Information Communication and Embedded Systems (ICICES), pp. 989-994, Sept. 2016.
- [39] U. Rathod, "Building a Smart Helmet with the Blues Wireless Notecard," Hackster.io, Jun. 2022.

[40] S. Kim, J. Lee, and H. Park, "Trends in Smart Helmets With Multimodal Sensing for Health and Safety: Scoping Review," *JMIR mHealth and uHealth*, vol. 10, no. 11, pp. e40797, Nov. 2022.

[41] A. D. T. Alcantara et al., "Internet of Things-Based Smart Helmet with Accident Identification and Logistics Monitoring for Delivery Riders," *Engineering Proceedings*, vol. 58, no. 1, pp. 129, Nov. 2023.

[42] A. Kumar, M. Bansal, and S. Gupta, "Implementation and Analysis of Smart Helmet," in *Proc. IEEE International Conference on Computing, Communication and Automation (ICCCA)*, pp. 1-4, 2017.

[43] "iC-R Smart Motorcycle Helmet," *Intelligent Cranium Helmets*, 2024-2025.

[44] "Sena's Phantom is a purpose-built, fully-integrated smart helmet," *New Atlas*, Mar. 2025.

[45] "Sena Motorcycles," *Apple App Store*, Jul. 2021.

[46] "Sena Cycling," *Google Play Store*.

[47] W. Waeosri, "Implement the Streaming Real-Time Java Application with Kotlin Language," *Medium*, Mar. 2022.

[48] "Remote Monitoring Mobile App Development with Kotlin," *Appilian*, Mar. 2025.

Appendix A

HelmX			
ORIGINALITY REPORT			
3%	1%	1%	1%
SIMILARITY INDEX	INTERNET SOURCES	PUBLICATIONS	STUDENT PAPERS
PRIMARY SOURCES			
1	Submitted to Higher Education Commission Pakistan Student Paper	<1%	
2	docslib.org Internet Source	<1%	
3	Submitted to National School of Business Management NSBM, Sri Lanka Student Paper	<1%	
4	www.coursehero.com Internet Source	<1%	
5	K.I. Wong, Yi-Chung Chen, Tzu-Chang Lee, Sheng-Min Wang. "Head Motion Recognition Using a Smart Helmet for Motorcycle Riders", 2019 International Conference on Machine Learning and Cybernetics (ICMLC), 2019 Publication	<1%	
6	Submitted to The Independent Institute of Education (IIE) Student Paper	<1%	
Submitted to University of Glamorgan			

